L'AUTOMATISATION COGNITIVE

DE LA DIMENSION SPATIALE

Une étude de la transition de la géomatique traditionnelle vers le développement assisté par l'intelligence artificielle

---

Méthodologie de conception de la Géomatique & IA agentique

Tristan VACHAUDEZ

Positionnement du travail

Ce manuscrit étudie la mise en place d'une architecture d'assistance puis par la suite replicant les capacités d'un géomaticien par la conception d'un assistant géomatique intégré à QGIS, associant un modèle de langage local, un système RAG fondé sur une documentation maîtrisée et un catalogue d'outils géomatiques déterministes (PyQGIS, GDAL, PostGIS). Il distingue explicitement l'état de l'art, la conception de l'architecture et le protocole d'évaluation empirique proposé. Les affirmations non vérifiables sont signalées par les marqueurs [À SOURCER] et [VÉRIFIER]. Il est ensuite mis en place une autonomie complète de l'intelligence artificielle via une architecture agentique.

1. Introduction : La mutation de l'analyse spatiale 6
2. Le cadre théorique : le modèle « 5-5-5 » et la trajectoire d'autonomie 6
2.1 Les cinq objectifs autonomes 7
2.2 La trajectoire d'autonomie (L1 à L5) 8
2.3 Problématique de recherche 9
2.4 Hypothèses de travail 9
2.5 Objectifs, périmètre et contributions 9
2.5.1 Objectifs 9
2.5.2 Périmètre 10
2.5.3 Contributions attendues 10
2.6 Plan du manuscrit 10
3 État de l'art : la géomatique à l'épreuve des agents autonomes 10
3.1 Des SIG vers les agents géospatiaux autonomes 10
3.2 Génération de code géospatial et pratiques de développement assisté 11
3.2.1 Structuration du projet et fichiers d'instructions 11
3.2.2 Méthodologie en deux phases 11
3.2.3 Standardisation des interfaces : le Model Context Protocol (MCP) 11
3.4.1 Ce que le MCP standardise et ce qui reste à développer 11
3.5 Interopérabilité spatiale : la convergence des standards 12
Exigences fonctionnelles et cadre d'évaluation 13
3.1 Cas d'usage et périmètre fonctionnel 13
3.2 Contraintes géomatiques et exigences de validité spatiale 13
3.3 Principes directeurs de conception 13
3.4 Critères de qualité et exigences fonctionnelles 14
3.5 Risques identifiés et lien avec les hypothèses 14
3.6 État des hypothèses au regard de l'état de l'art 14
4. Conception de l'architecture hybride 14
4.1 Deux architectures distinctes 14
4.2 Architecture A l'assistant géomatique intégré à QGIS 15
4.2.1 Rôles des modèles et clarification de la nomenclature 15
4.2.2 Le module RAG 16
4.2.3 La couche d'outils : frontière de sécurité 16
4.2.4 Validation déterministe et filtres d'intégrité géométrique 17
4.2.5 États d'une demande et observabilité 17
4.3 Architecture B : Runtime d'agent et contraintes matérielles 17
4.3.1 Contraintes matérielles : VRAM, KV Cache, quantification 17
4.3.2 Articulation avec la configuration du serveur 18
4.4 Intégration du plugin Assistant dans QGIS 18
4. Orchestration technique des flux de travail et collecte autonome de données 20
3.1 LLM-Find : l'approvisionnement autonome en données 20
4. Standardisation des interfaces de communication : le role du Model Context Protocol 22
5. Pratiques de codage assiste et rôle des IDE spécialisés 24
5.1 Compartiment des fichiers projet 25
5.2 Méthodologie en deux phases 25

6. Evaluation empirique, diagnostics d'hallucination et performances quantitatives..... 26
Méthodologie expérimentale proposée..... 29
5.1 Nécessité d'une évaluation propre au système..... 29
5.2 Constitution du corpus de tâches..... 30
5.3 Configurations comparées..... 30
5.4 Indicateurs de performance..... 30
5.5 Métriques formelles..... 31
5.6 Protocole de comparaison des sorties géométriques..... 31
5.7 Déroulement et reproductibilité..... 31
Résultats : benchmarks publiés et état de l'implémentation..... 33
6.1 État de l'art empirique : les benchmarks publiés..... 33
6.1.1 Analyse des mécanismes d'échec correction d'interprétation..... 33
6.2 État actuel de l'implémentation du prototype..... 33
7. Ethique de l'IA en géomatique..... 34
7.1 Les cadres de référence internationaux..... 34
7.2 Du principe éthique à la règle de droit : l'approche par les risques..... 34
8. L'impact sur l'environnement..... 35
8.1 La consommation d'électricité des centres de données..... 35
8.2 L'empreinte d'un usage : des ordres de grandeur très variables..... 35
8.3 L'analyse de cycle de vie de Mistral AI (juillet 2025)..... 35
9. Cadre réglementaire : AI Act et RGPD..... 36
Normalisation ISO, standards de l'OGC et gouvernance éthique..... 36
9.1 Union européenne : le règlement 2024/1689 (AI Act)..... 37
9.2 Les dispositions environnementales de l'AI Act..... 37
9.3 États-Unis..... 37
9.4 Points de vigilance sur la fiabilité..... 38
9.5 Souveraineté numérique et géospatiale..... 38
10. L'Émergence de l'IA Locale : Architecture, Performance et Souveraineté des Données en 2026..... 38
10.1. Introduction : Le Changement de Paradigme vers l'Inférence Locale..... 38
10.2. Analyse des Infrastructures Matérielles : Du Laptop au Data Center..... 39
Limites Critiques : VRAM et Cache KV..... 40
10.3. Écosystèmes Logiciels et Protocoles d'Installation..... 40
Protocoles de Déploiement Standardisés..... 40
Dualité Architecturale : Agentic vs Infrastructure..... 40
Deux philosophies dominent le marché :..... 40
4. Optimisation des Performances : Quantification et Gestion des Ressources..... 40
Variables d'Environnement Critiques pour la Production..... 41
5. Souveraineté des Données et Cadre Réglementaire (RGPD & AI Act)..... 41
Les avantages juridiques sont majeurs :..... 41
6. Impact Environnemental et Économique de l'IA Locale..... 41
Analyse du Coût Total de Possession (TCO)..... 41
Efficacité Énergétique..... 42
7. La spécialisation d'une IA locale..... 42
7.1 Architecture technique d'un Assistant IA intégré à QGIS..... 43
7.16 Vers une IA Autonome et Durable..... 63
11. L'automatisation ultime et la création d'une équipe IA..... 64

11.1 La Recette d'Établissement d'une Équipe de Géomaticiens Autonomes Propulsée par l'IA Agentique....64
11.1.1 Thématique & Cadre Stratégique de l'Autonomie....64
11.1.2 Le Socle Infrastructurel : Architecture Moteur & Runtime de l'Agent....66
11.1.3 L'Outillage Métier Géospatial & l'Intégration de l'agent....68
11.1.4 Cadre de Gouvernance Agentique, Sécurité & Contrôle d'Autonomie....69
11.1.5 Orchestration Multi-Agents, Automatisation Planifiée & Optimisation Économique....71
11.1.6. Feuille de Route Opérationnelle & Redéfinition du Rôle du Géomaticien....73
Conclusion....75
Redéfinition du rôle du géomaticien....75
Perspectives futures et recommandations operationnelles....75
perspectives d'ingénierie : Quatre axes de developpement operationnels....75
➔ Architectures hybrides : articuler l'assistant de bureau (A) et le service agentique (B) via une API, avec communication contrôlée....76
➔ Ingénierie logicielle rigoureuse : structures de répertoires standardisées, spécifications en amont, règles de style persistantes....76
➔ Filtres d'intégrité géométrique : renforcer les contrôles topologiques et CRS en amont du calcul géospatial....76
➔ Maintien de l'expertise humaine : conserver le regard critique du géomaticien comme pivot des processus décisionnels spatiaux....76
Vers l'architecture B service géomatique agentique....77
Bibliographie et référentiels....78
B.1 Publications et benchmarks académiques....78
B.2 Standards et protocoles....78
B.3 Référentiels normatifs et réglementaires....78
B.4 Concepts scientifiques....79
B.5 Outils et écosystèmes logiciels....79
B.6 Note sur la vérifiabilité....80
ANNEXES....81
Annexe 1: Architecture Multi IA....82
Architecture de l'assistant géomatique local dans QGIS : modèles, RAG et outils....82
A.1 Vue d'ensemble....82
A2. Le rôle de chaque modèle....82
A.3 Le module RAG....83
Exemple concret....84
« Quelles vérifications faut-il faire avant d'importer un réseau d'eau dans EPANET ? »....84
A.4 La couche d'outils....84
Exemple : retrouver une couche malgré une faute....85
A.5 Parcours d'une demande....86
Parcours A Opération simple....86
Parcours B Question ou opération complexe....86
A.6 Localisation des données et des traitements....86
A.7 Synthèse de l'architecture actuelle....87
Annexe 2 scénario d'exécution et pseudo-flux d'orchestration....88

1. Introduction : La mutation de l'analyse spatiale

La géomatique traditionnelle traverse une restructuration fondamentale de ses modèles d'interaction et de traitement sous l'impulsion des agents autonomes et des modèles de langage de grande taille (LLM). Historiquement, la pratique des systemes d'information geographique (SIG) s'est construite autour de logiciels de bureau lourds et d'environnements de programmation de scripts complexes, a l'instar d'ArcPy ou PyQGIS, necessitant une double expertise technique et thématique. Cette exigence imposait une charge cognitive importante aux professionnels, qui devaient traduire manuellement des concepts spatiaux abstraits en une série d'opérations géométriques au sein d'interfaces graphiques denses.

L'avènement de l'ingénierie géospatiale agentique redéfinit ce cadre en substituant l'interface utilisateur graphique traditionnelle par une interface en langage naturel. Le SIG autonome s'établit comme un partenaire actif dans la prise de décision, capable d'interpréter des requêtes descriptives de haut niveau pour concevoir, exécuter et valider des flux de traitement géospatiaux complexes sans intervention humaine directe. Le LLM fait office de centre de planification cognitive, tandis que des programmes exécutables principalement en Python agissent comme des effecteurs numériques au sein de l'environnement de données.

Cette transition repose sur la simplification des concepts géomatiques fondamentaux : projection cartographique, relations topologiques, calculs de zones tampons ou jointures spatiales sont encapsulés par les agents autonomes. Le choix d'un système de coordonnées de référence (CRS) approprié opération critique et sujette aux erreurs d'échelle et de distorsion est automatisé par l'agent, qui interroge des bases de connaissances géographiques pour projeter dynamiquement les données avant toute mesure de distance ou de surface.

> *L'utilisateur n'a plus à appréhender la syntaxe rigide des algorithmes ; il formule son intention scientifique, et l'architecture agentique traduit cette intention en un pipeline de traitement géographiquement cohérent et scientifiquement valide.*

2. Le cadre théorique : le modèle « 5-5-5 » et la trajectoire d'autonomie

Pour formaliser le développement des systèmes géospatiaux autonomes, la recherche académique a structuré le comportement et l'architecture de ces outils autour du modèle conceptuel dit « 5-5-5 ». Ce cadre organise la transition technologique selon trois axes interdépendants : les objectifs d'autonomie comportementale, les niveaux de progression de l'indépendance du système, et les fonctions architecturales requises pour l'orchestration logicielle.

Le cadre « 5-5-5 » mobilisé dans ce manuscrit s'inspire directement du framework de l'Autonomous GIS proposé par Feng, McDonald et Zhang, qui définit cinq objectifs autonomes (self-generating, self-organizing, self-verifying, self-executing, self-growing), cinq niveaux d'autonomie (de Routine-aware à Knowledge-aware GIS) et cinq fonctions architecturales. Nous l'adaptons au contexte

de la géomatique assistée par LLM et l'utilisons comme fil directeur de la problématique. [Source : Feng, S., McDonald, K. A., & Zhang, L. (2024/2025). GIScience in the era of Artificial Intelligence: A Research Agenda towards Autonomous GIS.]

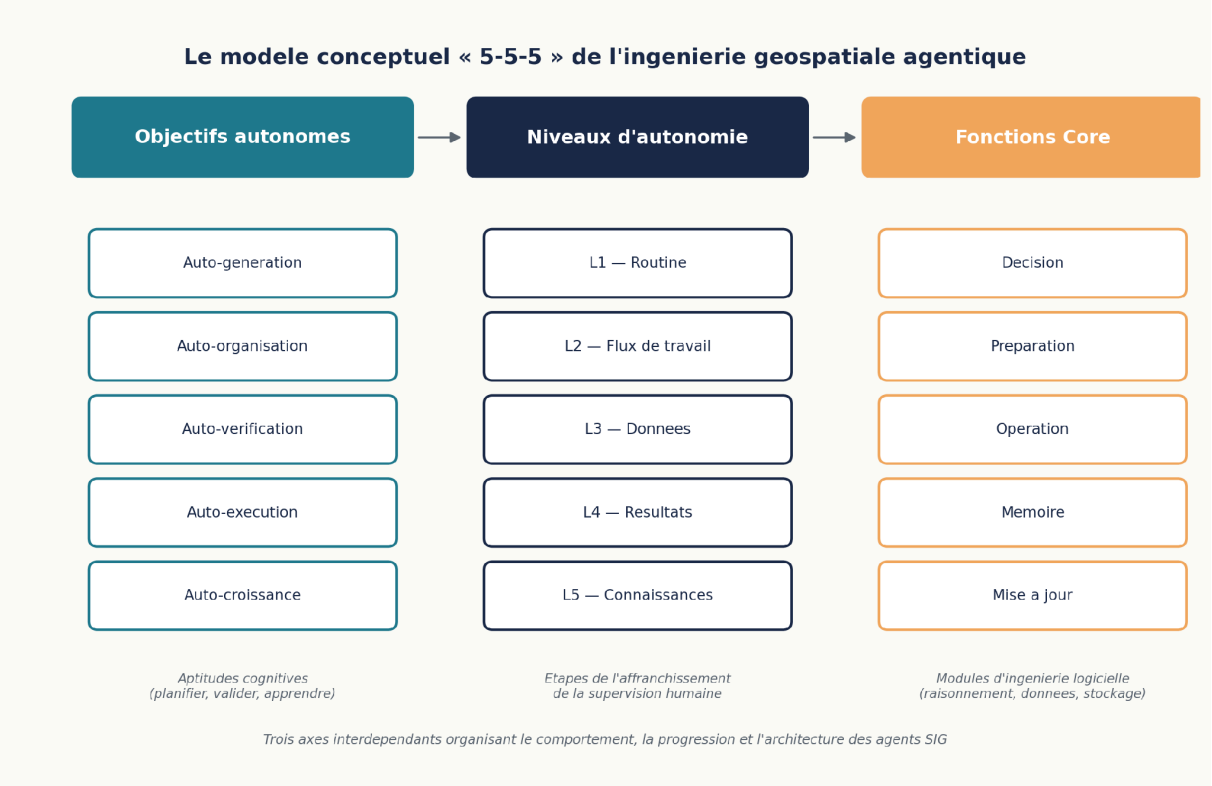


Figure 1 Les trois axes interdépendants du modèle 5-5-5

2.1 Les cinq objectifs autonomes

➔ Auto-generation : concevoir de manière indépendante des hypothèses et stratégies d'analyse spatiale.

➔ Auto-organisation : ordonner les ressources de calcul et structurer les modules de code en flux logiques.

➔ Auto-vérification : contrôler l'intégrité topologique et la cohérence géographique des données produites.

➔ Auto-exécution : traduire physiquement les modèles en cartes, graphiques ou rapports analytiques.

➔ Auto-croissance : apprendre des échecs, assimiler de nouvelles API et enrichir la bibliothèque de fonctions réutilisables.

Les cinq objectifs autonomes guident le comportement de l'agent face à un problème spatial indéterminé. L'auto-génération concerne l'aptitude du système à concevoir de manière indépendante des hypothèses et des stratégies d'analyse spatiale. L'auto-organisation permet à

l'agent d'ordonner ses ressources de calcul et de structurer ses modules de code sous la forme de flux logiques. L'auto-vérification intervient comme un mécanisme de contrôle qualité interne, où l'agent teste l'intégrité topologique et la cohérence géographique des données produites. L'auto-exécution assure la traduction physique des modèles en cartes, graphiques ou rapports analytiques. Enfin, l'auto-croissance décrit la capacité du système à apprendre de ses échecs, à assimiler de nouvelles documentations d'API et à enrichir sa bibliothèque interne de fonctions réutilisables.

2.2 La trajectoire d'autonomie (L1 a L5)

La trajectoire d'autonomie s'articule en cinq niveaux progressifs, du script statique a la cognition spatiale totale.

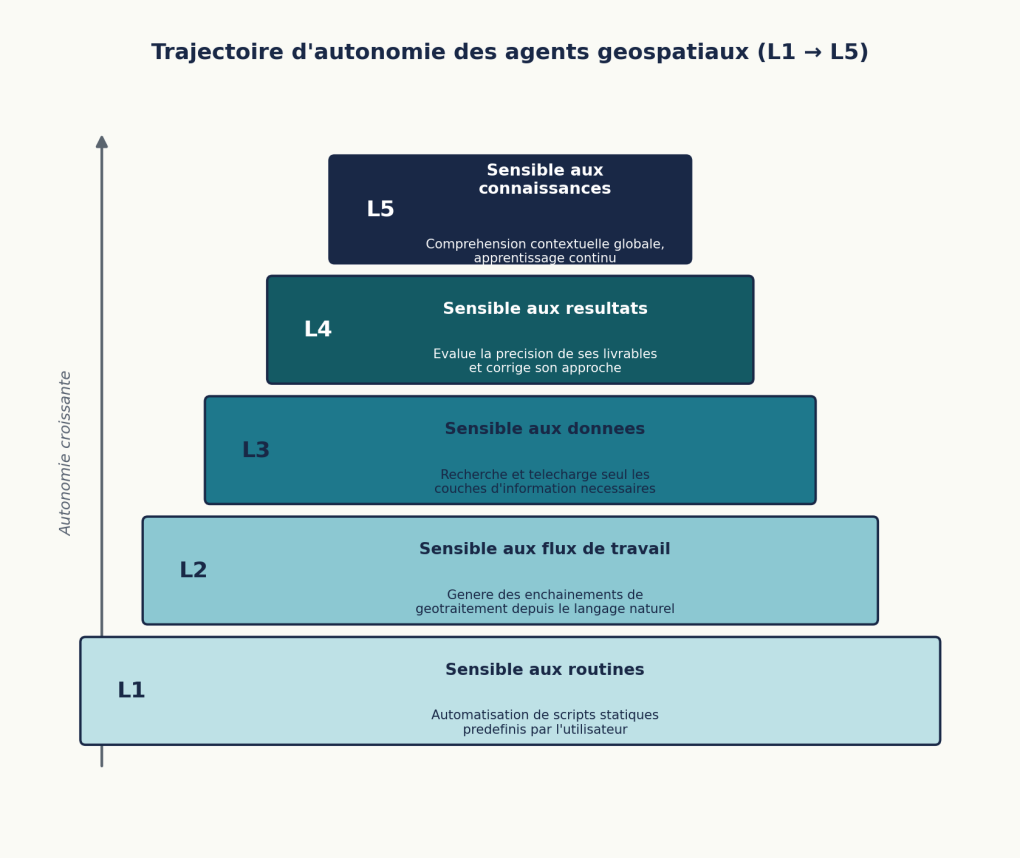


Figure 2 Progression des cinq niveaux d'autonomie

La trajectoire d'autonomie s'articule quant à elle en cinq niveaux progressifs. Le niveau initial, qualifié de système sensible aux routines (L1), correspond à l'automatisation classique de tâches répétitives via des scripts statiques pré-définis par l'utilisateur. Le niveau sensible aux flux de travail (L2) introduit la capacité de générer dynamiquement des enchaînements de géotraitement à partir de requêtes en langage naturel, bien que l'agent reste tributaire de l'humain pour la préparation des données. Le niveau sensible aux données (L3) marque une rupture majeure en dotant l'agent de la capacité de rechercher, filtrer et télécharger de manière indépendante les couches d'informations géographiques nécessaires auprès de sources en ligne. Le niveau sensible aux

résultats (L4) permet au système d'évaluer de façon critique la précision de ses livrables et de modifier son approche algorithmique en cas d'erreur de modélisation. Le niveau sensible aux connaissances (L5) représente le stade ultime d'intégration, où l'agent fait preuve d'une compréhension contextuelle globale et résout des problématiques spatiales complexes en mobilisant des modèles d'apprentissage continu.

2.3 Problématique de recherche

L'accumulation de technologies pertinentes, agents autonomes, QGIS, PyQGIS, RAG, Model Context Protocol, modèles locaux, validation géométrique, orchestration multi-agents ne constitue pas en elle-même un travail de recherche. Celui-ci exige l'identification d'un problème non résolu, la clarification de ce qui est déjà établi dans la littérature, et la définition précise de l'apport du travail. La question à laquelle ce manuscrit cherche à répondre est la suivante :

Problématique

Dans quelle mesure une architecture hybride associant un modèle de langage local, un système RAG fondé sur une documentation maîtrisée et des outils géomatiques déterministes permet-elle d'automatiser des traitements SIG formulés en langage naturel, tout en garantissant la validité spatiale, la traçabilité et le contrôle des opérations ?

Cette formulation circonscrit le sujet à une architecture identifiable, fait ressortir les difficultés scientifiques centrales (validité spatiale, traçabilité, contrôle) et ouvre directement sur une évaluation empirique.

2.4 Hypothèses de travail

Trois hypothèses guident la conception et l'évaluation. Il s'agit de propositions à vérifier, et non de résultats acquis :

H1 Fiabilité

La validation déterministe des paramètres et des opérations réduit le nombre de traitements incorrects par rapport à une approche reposant uniquement sur le modèle de langage.

H2 Ancrage documentaire

L'ajout d'un RAG fondé sur des sources vérifiées améliore la pertinence des méthodes proposées et la qualité des références fournies.

H3 Efficacité

Le routage entre un modèle léger pour les opérations simples et un modèle plus performant pour les opérations complexes réduit le coût de calcul ou le temps de réponse, sans dégrader excessivement le taux de réussite des tâches.

2.5 Objectifs, périmètre et contributions

2.5.1 Objectifs

- Concevoir une architecture d'assistance géomatique intégrée à QGIS, articulée autour d'un LLM local, d'un RAG documentaire et d'outils déterministes.
- Distinguer explicitement, dans le manuscrit, l'état de l'art, la conception technique et l'évaluation empirique.
- Proposer un protocole d'évaluation reproductible permettant de mesurer l'apport réel de l'architecture.

2.5.2 Périmètre

Le travail se concentre sur l'automatisation de traitements SIG vectoriels et attributaires courants (tampons, reprojections, intersections, jointures spatiales, contrôle de géométries, production de couches) dans

l'environnement QGIS. Il ne couvre pas la formation de modèles de deep learning, ni l'analyse d'images satellitaires à grande échelle, ni le déploiement multi-sites à très grande échelle, ceux-ci sont abordés comme perspectives.

2.5.3 Contributions attendues

- **Une architecture hybride concrète** et documentée, reliant un LLM local, un RAG maîtrisé et des outils PyQGIS/GDAL/PostGIS avec validation déterministe.
- **Une clarification conceptuelle** opposant planification cognitive et exécution déterministe, et distinguant deux architectures (bureau vs service).
- **Un protocole d'évaluation** graduel, comparant plusieurs configurations sur un corpus de tâches géomatiques représentatives.

2.6 Plan du manuscrit

Le document est organisé en quatorze chapitres. Le chapitre 1 présente la mutation paradigmatique de l'analyse spatiale. Le chapitre 2 pose le cadre théorique (modèle « 5-5-5 », trajectoire d'autonomie, problématique et hypothèses). Le chapitre 3 dresse l'état de l'art des agents géospatiaux, de la génération de code géospatial, du RAG et des protocoles d'interopérabilité. Le chapitre 4 définit les exigences fonctionnelles et le cadre d'évaluation. Le chapitre 5 présente la conception de l'architecture hybride. Le chapitre 6 décrit les pratiques de codage assisté. Le chapitre 7 rend compte de l'état de l'art empirique (benchmarks). Le chapitre 8 détaille la méthodologie expérimentale proposée. Le chapitre 9 discute les résultats et l'état de l'implémentation. Le chapitre 10 aborde l'éthique, 11 l'environnement et 12 le cadre réglementaire. Le chapitre 13 traite de l'émergence de l'IA locale. Le chapitre 14 examine l'automatisation ultime et la création d'une équipe IA. Les annexes concluent sur l'architecture multi-IA et un scénario d'exécution.

3 État de l'art : la géomatique à l'épreuve des agents autonomes

3.1 Des SIG vers les agents géospatiaux autonomes

L'intégration des modèles de langage dans les systèmes d'information géographique ouvre de nouvelles perspectives d'automatisation des traitements spatiaux. En interprétant des demandes formulées en langage naturel, un agent peut contribuer à la préparation de workflows, à la sélection d'outils de géotraitement et à l'exploitation de documentation technique. Le prototype LLM-Geo illustre cette voie : un noyau décisionnel basé sur un LLM orchestre des calculs de bout en bout, en convertissant la demande textuelle de l'utilisateur en un graphe orienté acyclique (DAG) de géotraitement. Chaque nœud correspond à une opération unitaire intersection spatiale, re-projection, agrégation statistique tandis que les arêtes définissent les dépendances et le sens de circulation des flux d'informations géographiques.

> **Lecture critique**
> Les architectures agentiques promettent une autonomie croissante, mais les benchmarks actuels montrent que le taux de réussite des tâches complexes demeure limité (voir chapitre 6). L'état de l'art doit donc être lu avec prudence : la capacité à produire un pipeline plausible ne se confond pas avec sa fiabilité opérationnelle.

3.2 Génération de code géospatial et pratiques de développement assisté

La génération de code par les LLM bouleverse la production logicielle en géomatique. Les environnements de développement assistés par l'IA (Cursor, Claude Code, GitHub Copilot, Windsurf) ne se limitent plus à la suggestion de portions de code isolées : ils appréhendent l'architecture globale d'un projet, coordonnent des modifications simultanées sur plusieurs scripts Python et exécutent des tâches en arrière-plan via des agents autonomes.

3.2.1 Structuration du projet et fichiers d'instructions

Pour prévenir la dégradation qualitative du code généré, les professionnels appliquent une méthodologie structurée, autour d'un répertoire dédié et de fichiers d'instructions persistantes :

Tableau 2.1 Structuration d'un projet géomatique assisté par IA.

Ressource	Rôle
/src	Code source géomatique.
/docs	Spécifications techniques (cahier des charges).
/tests	Tests unitaires et d'intégration.
/reference	Gabarits de développement validés par l'entreprise.
CLAUDE.md / .cursor/rules/ / .github/copilot-instructions.md	Fichiers d'instructions persistantes : identité du projet, versions d'API, contraintes d'importation, conventions.

3.2.2 Méthodologie en deux phases

La phase préparatoire commence par la co-conception d'un cahier des charges détaillé (stocké sous /docs) définissant l'architecture logicielle, les formats de données acceptés, les critères d'acceptation fonctionnels et les éléments explicitement exclus afin d'éviter la dérive fonctionnelle. La phase de construction procède par blocs restreints et testables, sous contrôle d'un système de versions (Git), offrant un filet de sécurité permettant de restaurer un état stable lorsque l'agent produit du code non-exécutable. Les revues de code sont privilégiées via des sessions d'agents indépendantes, disposant d'un historique vierge de biais méthodologiques.

3.2.3 Standardisation des interfaces : le Model Context Protocol (MCP)

L'intégration industrielle des agents au sein d'infrastructures d'entreprise s'est accélérée grâce à l'émergence du Model Context Protocol (MCP) expliqué au chapitre 4, un standard ouvert introduit par Anthropic à la fin de l'année 2024. Conçu comme une couche d'abstraction entre les IA génératives et les systèmes externes, le MCP fournit une sémantique normalisée permettant à un agent de découvrir les capacités d'un outil, d'en comprendre les paramètres d'appel et d'en consommer les résultats.

3.4.1 Ce que le MCP standardise et ce qui reste à développer

Il importe de ne pas présenter le MCP comme une couche universelle supprimant toute nécessité de développement d'intégration. Le protocole standardise le contrat d'exposition et d'appel des outils ; il ne dispense en revanche pas de développer et de sécuriser chaque serveur métier, ni de garantir la validité sémantique et topologique des opérations géomatiques exécutées.

Dans le domaine de la géomatique, ce protocole unifie l'accès aux progiciels de cartographie et aux bases de données géographiques. Des outils comme le serveur open-source gis-mcp exposent directement aux modèles de langage les capacités analytiques de bibliothèques fondamentales (Shapely, PyProj, Rasterio,

PySAL), permettant aux agents de s'appuyer sur des fonctions mathématiques rigoureuses plutôt que sur leur propre calcul, intrinsèquement peu fiable pour la géométrie.

Tableau 2.2 Exemples de serveurs MCP appliqués à la géomatique.

Serveur MCP	Capacités exposées	Environnement d'exécution
gis-mcp	Shapely, PyProj, Rasterio, PySAL	Local / serveur métier
CARTO MCP	200+ composants d'analyse géospatiale (carto sql query, workflows components/validate/create)	Cloud entreprise (BigQuery, Snowflake, Databricks)
Google Maps MCP	text_query, resolve_maps_urls, view_map	Cloud Google

L'architecture cloud s'est également emparée de cette technologie avec le serveur CARTO MCP, qui permet d'orchestrer plus de deux cents composants d'analyse géospatiale directement au sein d'entrepôts de données, le traitement s'exécutant dans le cloud de l'entreprise et garantissant sécurité et traçabilité. Du côté des services de géolocalisation commerciaux, le serveur MCP de Google Maps intègre des outils spécialisés : la recherche textuelle géographique (text_query, avec un paramètre location_bias), la résolution d'adresses ambiguës (resolve_maps_urls, convertissant les URL de partage en Place IDs) et l'affichage cartographique interactif (view_map), permettant au concepteur humain de valider visuellement la géométrie produite.

Statut de la source

La date d'introduction du MCP (fin 2024) et le nombre de composants CARTO MCP (« plus de 200 ») doivent être confirmés par la documentation officielle au moment de la soutenance [VÉRIFIER]. Les noms de serveurs (gis-mcp, CARTO MCP, Google Maps MCP) et de leurs outils doivent être rattachés à leurs dépôts respectifs [À SOURCER].

3.5 Interopérabilité spatiale : la convergence des standards

Les standards de l'Open Geospatial Consortium (OGC) WMS, WFS, WCS, GeoJSON constituent le socle de l'interopérabilité spatiale. L'alimentation des pipelines géomatiques automatisés requiert une adhésion stricte à ces standards, assurant une structuration uniforme lors de l'acquisition autonome de données. La convergence entre le MCP (contrat d'appel des outils) et les standards OGC (contrat des données) est ce qui permet, en théorie, à un agent de découvrir, d'appeler et d'interpréter des services géospatiaux hétérogènes de manière fiable.

Synthèse de l'état de l'art

L'état de l'art révèle un écosystème riche mais immature : les agents géospatiaux démontrent des capacités prometteuses de planification, mais peinent encore sur la fiabilité exécutoire (chapitre 6). Les standards (MCP, OGC) fournissent le socle d'interopérabilité, mais n'éliminent pas la nécessité de validations déterministes spécifiques au domaine géomatique nécessité au cœur de l'architecture proposée au chapitre 4.

Exigences fonctionnelles et cadre d'évaluation

3.1 Cas d'usage et périmètre fonctionnel

L'assistant géomatique vise à fournir une interface conversationnelle capable de relier les données visibles dans QGIS à des connaissances documentaires et à des opérations géospatiales explicables. Les objectifs fonctionnels sont les suivants :

- Comprendre une demande formulée en langage naturel et identifier les couches, emprises, attributs et opérations concernés.
- Rechercher les méthodes, règles, procédures et scripts pertinents au moyen du module RAG.
- Planifier une séquence d'actions géospatiales avant toute exécution.
- Valider les paramètres, les permissions, les contraintes de ressources et les effets attendus.
- Exécuter les outils autorisés via les interfaces prévues par l'architecture.
- Restituer les résultats, les actions réalisées, les avertissements et les sources dans QGIS.

Le périmètre couvre les opérations SIG vectorielles et attributaires courantes : tampons, reprojections, intersections, sélection spatiale, jointures, contrôle de qualité et production de couches dérivées. Les exemples qui suivent n'impliquent la présomption d'aucune donnée particulière, d'aucun volume, d'aucune coordonnée ni d'aucun résultat réel : les résultats attendus sont des types de restitution à produire après validation et exécution.

3.2 Contraintes géomatiques et exigences de validité spatiale

Les traitements SIG reposent sur des contraintes spécifiques que l'automatisation ne peut ignorer :

Tableau 3.1 Contraintes géomatiques et risques associés.

Contrainte	Enjeu	Risque en cas de défaut
Cohérence des CRS	Toutes les couches impliquées doivent partager un système de coordonnées compatible.	Intersections vides, zones tampons incorrectes, mesures faussées.
Validité des géométries	Les polygones, lignes et points doivent respecter les règles topologiques.	Calculs erronés, échecs d'algorithmes, livrables non conformes.
Unités de mesure	Distances, surfaces et volumes doivent être exprimés dans l'unité attendue.	Erreurs d'échelle (m vs m² vs degrés).
Compatibilité des données	Schémas, types de géométrie et noms de champs cohérents.	Jointures cassées, perte d'attributs.
Préservation des attributs	Les métadonnées et champs sémantiques doivent être conservés.	Perte d'information, décisions sur données appauvries.

Une erreur de planification ou une hypothèse implicite peut conduire à un résultat techniquement exploitable en apparence, mais spatialement incorrect. La sécurité et la traçabilité deviennent déterminantes lorsque les agents interagissent avec des données professionnelles ou des infrastructures géographiques sensibles.

3.3 Principes directeurs de conception

- **Traçabilité :** chaque action est journalisée avec ses paramètres, son statut et ses références.
- **Validation humaine conditionnée :** le risque de l'action détermine le niveau de confirmation requis.
- **Moindre privilège :** l'agent ne dispose que des droits strictement nécessaires à sa mission.

- **Explicabilité :** les opérations sont restituées de manière compréhensible pour un géomaticien.
- **Séparation stricte :** la génération de texte est dissociée de l'exécution d'outils.

3.4 Critères de qualité et exigences fonctionnelles

Pour chaque cas, l'assistant doit distinguer la demande initiale, les hypothèses confirmées, les données réellement utilisées, la méthode appliquée, le résultat produit, les avertissements et les sources documentaires citées. Si la demande ne précise pas un seuil, une emprise, une unité ou une règle métier, le système doit demander une confirmation ou proposer un plan non exécuté. Il ne doit pas compléter silencieusement ces paramètres à partir d'une supposition.

3.5 Risques identifiés et lien avec les hypothèses

L'analyse des risques structure la conception. Le tableau suivant relie les risques techniques aux hypothèses de recherche (H1–H3, chapitre 1) et aux contrôles prévus :

Tableau 3.2 Matrice risques / contrôles / hypothèses.

Risque	Cause sous-jacente	Contrôle prévu	Hypothèse
Traitement incorrect	Erreur de planification ou de paramétrage par le LLM	Validation déterministe des paramètres et CRS	H1
Réponse non fondée	Hallucination documentaire	RAG sur sources maîtrisées + citations	H2
Coût/latence excessifs	Sollicitation systématique du modèle lourd	Routage modèle léger/lourd	H3
Action non autorisée	Surautorisation de l'agent	Moindre privilège + file d'approbation	H1
Sortie géométrique fausse	Absence de contrôle topologique	Filtre d'intégrité géométrique	H1

3.6 État des hypothèses au regard de l'état de l'art

Les benchmarks publiés (chapitre 6) montrent que les défaillances de planification logique et les erreurs d'alignement géométrique (CRS) sont les principales causes d'échec des agents géospatiaux. Ces constats motivent directement l'hypothèse H1 : l'apport d'une couche déterministe de validation devrait réduire significativement le nombre de traitements incorrects. L'hypothèse H2 repose sur l'idée que l'ancrage dans une documentation maîtrisée limite l'hallucination méthodologique. L'hypothèse H3 s'appuie sur le routage de modèles pour concilier performance et coût. Aucune de ces hypothèses n'est présumée acquise : elles font l'objet du protocole d'évaluation défini au chapitre suivant.

4. Conception de l'architecture hybride

4.1 Deux architectures distinctes

L'analyse du document initial révèle une contradiction structurelle : l'IA locale est défendue pour la confidentialité et la souveraineté, tandis qu'une autre section privilégie un agent persistant hébergé sur serveur ou VPS, avec conteneurisation et disponibilité permanente. Ces deux approches ne sont pas incompatibles, mais elles répondent à des besoins différents et ne doivent pas être présentées comme une seule architecture. Ce manuscrit en distingue explicitement deux :

Tableau 4.1 Distinction entre l'assistant de bureau (A) et le service agentique (B).

Critère	Architecture A Assistant SIG de bureau	Architecture B Service géomatique agentique
Finalité	Assister directement le géomaticien dans QGIS	Automatiser des tâches longues, planifiées ou concurrentes
Hébergement	Poste de travail local, modèles locaux	Serveur / VPS conteneurisé (Docker), disponible 24/7
Modèle	LLM local (Ollama)	LLM local ou distant via passerelle
Contrôle	Confirmation humaine par défaut	Files d'approbation, journaux, supervision
Données	Restent sur la machine de l'utilisateur	Confinees dans le conteneur, réseau isolé
Usage typique	Requête interactive, exploration	Traitements batch, surveillance, alertes

L'Architecture A est celle du prototype présenté au chapitre 5 et évalué au chapitre 6. L'Architecture B est décrite comme une évolution possible, replacée dans les perspectives du chapitre 8. Les deux peuvent communiquer via une API, mais leurs compromis respectifs doivent être explicites.

Convention de statut des composants

Chaque composant de l'architecture est attribué d'un statut explicite tout au long de ce chapitre : [Existant et testé], [En cours de développement], [Proposé pour l'architecture cible], ou [À évaluer]. Un fonctionnement envisagé n'est jamais présenté comme un résultat expérimental.

4.2 Architecture A l'assistant géomatique intégré à QGIS

L' Assistant géomatique IA est une architecture d'assistance géospatiale conçue pour interpréter des demandes en langage naturel, exploiter le contexte documentaire pertinent, planifier des opérations spatiales contrôlées et restituer les résultats dans QGIS. Le schéma de référence confirme les éléments suivants : QGIS fournit les couches Bâtiments, Routes et Cours d'eau ; l'assistant s'appuie sur des modèles locaux (famille Gemma) ; un module RAG exploite la documentation, les méthodes et les scripts ; une couche d'outils géospatiaux exécute des opérations au moyen de PyQGIS, GDAL et PostGIS ; les résultats et les actions sont renvoyés dans QGIS.

Statut des composants

Les composants sont décrits comme prévus par l'architecture ou en cours de développement ; les mécanismes de stockage, d'indexation, d'exécution et d'hébergement restent des choix d'implémentation à confirmer [VÉRIFIER].

4.2.1 Rôles des modèles et clarification de la nomenclature

L'architecture n'assigne pas à un seul modèle la tâche de tout faire. Chaque composant a un rôle précis. La nomenclature a été harmonisée pour lever les incohérences du document initial (Gemma 12B dans le corps vs Gemma 4B / EmbeddingGemma / Gemma-Geo en annexe) : il s'agit d'un routage multi-modèle, et non de versions successives contradictoires.

Tableau 4.2 Rôles et statuts des modèles. Les noms de modèles exacts sont à confirmer.

Modèle (référence locale)	Rôle	Statut
Gemma 4b : Modèle d'analyse rapide (famille Gemma, léger)	Extraction de l'opération et des paramètres pour les tâches simples	[En cours de développement]
EmbeddingGemma : modèle entrainé	Vectorisation des textes pour la recherche sémantique du RAG	[Proposé]
Modèle de raisonnement (Gemma-Geo ou équivalent)	Raisonnement et plan d'action pour les tâches complexes	[À évaluer]

Statut de la source
Les noms de modèles exacts (Gemma 4B, Gemma-Geo, EmbeddingGemma) et leurs versions doivent être confirmés par rapport aux dépôts Ollama/Hugging Face au moment de la soutenance [VÉRIFIER]. Le choix d'un routage multi-modèle doit être justifié par des mesures de coût/latence/précision (chapitre 5).

4.2.2 Le module RAG

Le RAG (Retrieval-Augmented Generation) sert à fournir à l'assistant un contexte vérifiable provenant de sources maîtrisées. Il ne doit pas être considéré comme une preuve automatique de validité : la qualité de la réponse dépend de la qualité, de l'actualité et du périmètre des sources récupérées. La chaîne de traitement comprend :

1. **Ingestion** recueillir la documentation, procédures, méthodes et scripts autorisés, avec métadonnées, statut, propriétaire et date de révision.
2. **Normalisation** extraire le texte et les éléments structurés utiles, préserver titres et avertissements, écarter les contenus non autorisés.
3. **Découpage** segmenter les sources en unités cohérentes ; taille et recouvrement à évaluer sur des exemples représentatifs.
4. **Enrichissement** associer domaine, méthode, type d'outil, statut, version et restrictions d'usage.
5. **Indexation** recherche textuelle, sémantique, ou combinaison (EmbeddingGemma).
6. **Recherche** transformer la demande en requête, appliquer les filtres d'autorisation.
7. **Classement et filtrage** ordonner, éliminer les sources incompatibles ou obsolètes, limiter le contexte transmis.
8. **Génération** demander au modèle de répondre à partir du contexte, en séparant faits sourcés, hypothèses et plan d'action.
9. **Citation** retourner les références des sources utilisées avec précision suffisante pour vérification humaine.

Gouvernance des connaissances
Les sources du RAG doivent avoir un propriétaire, un statut, une date de révision et un périmètre d'utilisation. Une méthode ou un script retiré du registre ne doit plus être proposé pour une nouvelle exécution, même s'il reste présent dans un index ou un historique.

4.2.3 La couche d'outils : frontière de sécurité

La couche d'outils est une frontière de sécurité entre le raisonnement génératif et les opérations géospatiales. Les outils sont exposés par un catalogue explicite, avec des paramètres déclarés, des préconditions, des sorties attendues et des règles d'autorisation. Le principe général : le modèle ne modifie pas directement le projet QGIS ; il interprète une demande et propose une action structurée ; le code Python appelle ensuite les fonctions réelles.

Un outil fondamental illustre ce mécanisme : la recherche de couches malgré une faute de frappe. Lorsque l'utilisateur écrit « Fais un buffer de 100 m autour de l'assainissement », le modèle extrait le nom fourni, puis une fonction de recherche compare ce nom aux couches du projet (par ex. via difflib.get_close_matches) pour récupérer le nom exact, et prépare une action structurée :

{"type": "action", "action": "buffer", "parameters": {"layer": "<nom_exact>", "distance": 100, "segments": 5, "dissolve": false, "output": "buffer_<nom_exact>"}}

Ce dictionnaire est un contrat entre l'analyse de l'IA et le code Python. Ses paramètres doivent être vérifiés avant l'exécution : l'IA peut extraire une mauvaise distance ou se tromper sur la couche. Attention toutefois

: ce mécanisme est un système d'outils Python piloté par le modèle, et non nécessairement un tool calling natif où le modèle choisit et appelle lui-même des fonctions via un protocole dédié.

Les contrôles spécifiques par interface : PyQGIS (limiter les classes, traitements et opérations accessibles au catalogue approuvé) ; GDAL (contrôler les formats, chemins, pilotes, emprises, tailles de sortie et options modifiant des données) ; PostGIS (requêtes paramétrées ou opérations préenregistrées, contrôle des schémas accessibles, empêcher les instructions non autorisées).

4.2.4 Validation déterministe et filtres d'intégrité géométrique

La validation des paramètres précède toute exécution : vérification des types, des plages, des noms de couches, des systèmes de coordonnées, des emprises, des volumes, des destinations et des droits. Une confirmation humaine est demandée lorsque l'action modifie des données, produit un volume important, touche une source sensible ou présente un risque difficilement réversible. Les filtres d'intégrité géométrique complètent cette validation en détectant précocement les anomalies de planification logique inversions d'opérations géométriques, absence de transformation de projection (CRS) avant toute consommation de ressources de calcul.

4.2.5 États d'une demande et observabilité

Les objets structurés versionnés sont privilégiés au texte libre lorsque l'échange déclenche une action. Les états possibles d'une demande sont : Reçue, À préciser, Planifiée, En attente de validation, En cours, Réussie avec avertissements, Échouée. Une erreur doit être associée à un statut explicite, un message compréhensible, un identifiant de corrélation et, lorsque cela est possible, une action corrective. Le système évite les reprises automatiques non bornées et ne relance pas une opération susceptible de produire des doublons sans confirmation.

4.3 Architecture B : Runtime d'agent et contraintes matérielles

L'architecture peut s'appuyer sur un runtime d'agent open-source (Hermes Agent, développé par Nous Research), qui s'exécute en tant que démon d'arrière-plan et expose une interface API HTTP compatible avec les standards de l'industrie. Trois points d'accès se distinguent : un point sans état (chat completions, compatible OpenAI), un point avec gestion d'état (responses, persistance de l'historique via previous_response_id), et un point asynchrone (runs, pour les tâches longues, avec interrogation d'état, abonnement aux événements, interruption et validation explicite).

> **Statut de la source**
> Les détails d'API (endpoints /v1/chat/completions, /v1/responses, /v1/runs, en-tête Idempotency-Key, en-tête X-Hermes-Session-Key, port par défaut) doivent être rattachés à la documentation technique officielle et à une version de référence explicite [À SOURCER]. Ces éléments sont susceptibles d'évoluer.

La mémoire persistante à long terme repose sur un cycle d'apprentissage fermé, couplant une base de recherche textuelle (SQLite FTS5) à un moteur de modélisation de l'utilisateur. Pour maintenir le cloisonnement strict des mémoires lors d'utilisations multi-utilisateurs ou partagées, le serveur s'appuie sur un en-tête HTTP spécifique (X-Hermes-Session-Key), qui définit l'identifiant stable de l'utilisateur ou du canal, distinct de l'en-tête de session courante (X-Hermes-Session-Id).

4.3.1 Contraintes matérielles : VRAM, KV Cache, quantification

La viabilité d'un projet d'IA locale repose sur l'adéquation entre les capacités de calcul et la taille des modèles. La variable limitante reste la VRAM. Un piège architectural courant est d'ignorer l'impact du Cache

KV : étendre la fenêtre de contexte de 4K à 32K tokens peut doubler les besoins en VRAM pour un même modèle. Si la mémoire vidéo sature, le système bascule sur le CPU, provoquant une chute de performance (de ~100 tokens/s à moins de 5 tokens/s).

Tableau 4.3 Niveaux de quantification GGUF / llama.cpp. Configuration de référence à documenter.

Quantification	VRAM (Llama 3.1 8B / RTX 4090)	Débit	Précision
FP16 (non quantifié)	~14 Go	~15 tok/s	100 %
Q8_0 (8 bits)	~9 Go	~28 tok/s	~99 %
Q4_K_M (4 bits standard)	~6 Go	95–110 tok/s	~95 %

Statut de la source
Les chiffres de VRAM, débit et précision sont indicatifs d'une configuration de référence (RTX 4090, Llama 3.1 8B). Ils doivent être rattachés à la source du benchmark, à la date d'observation et aux paramètres exacts [VÉRIFIER]. La quantification Q4_K_M est présentée comme standard industriel, mais son adéquation au domaine géomatique doit être validée par l'évaluation [À SOURCER].

Pour garantir la stabilité en environnement multi-utilisateurs, l'expert doit configurer des variables d'environnement critiques : activation de l'attention flash (gain de vitesse), nombre de requêtes traitées simultanément, et limitation du nombre de modèles résidents en VRAM pour éviter les collisions mémoire.

4.3.2 Articulation avec la configuration du serveur

Les contraintes matérielles s'articulent directement avec la configuration du serveur d'agent. Le risque majeur d'explosion des coûts et des performances en géomatique découle de la taille des données spatiales renvoyées par les outils : une seule requête renvoyant des entités GeoJSON complexes peut générer plusieurs mégaoctets de texte, qui, conservés verbatim dans la base d'historique, seraient réinjectés à chaque tour et provoqueraient une saturation mémoire. Le garde-fou consiste à plafonner la taille des sorties d'outils stockées dans l'historique (par exemple à 2 000 caractères), ce qui tronque les sorties volumineuses pour les tours suivants sans altérer le résultat immédiat ni l'exécution réelle.

Ce plafonnement s'applique exclusivement à la mémoire d'historique, en conservant le début de la réponse suivi d'une mention de troncature. Ce garde-fou garantit que la base de données d'historique et la fenêtre de contexte restent d'une taille optimale lors des chaînages conversationnels longs.

4.4 Intégration du plugin Assistant dans QGIS

L'intégration dans QGIS repose sur un script pilote qui orchestre les étapes, récupère les couches et choisit le traitement adapté. L'interface expose un bouton d'exécution qui peut demander confirmation avant de lancer le traitement Processing, QGIS créant alors le résultat et l'ajoutant au projet. Le contexte QGIS (couches présentes, noms, types de géométrie, CRS) est transmis à l'assistant de manière filtrée selon la demande, afin d'éviter d'envoyer des milliers de couches inutiles au modèle.

Pour gérer les réponses ambigües (« 100 m », « prends la deuxième couche »), il convient de conserver l'historique des échanges dans l'interface QGIS et de réinjecter ce contexte dans l'analyse suivante. Lorsque des informations manquent par exemple la distance d'un buffer, le système pose une question à l'utilisateur au lieu de laisser le modèle inventer une valeur.

Synthèse de la conception
L'architecture A est une IA locale hybride avec RAG et outils : les modèles comprennent et raisonnent, le RAG apporte des

connaissances documentaires, et Python garde la maîtrise des interactions avec QGIS. La validité géométrique et la sécurité sont assurées par des contrôles déterministes externes au LLM, conformément à l'hypothèse H1.

L'architecture B est un assistant de niveau expert qui est autonome sur une tâche de A à Z. Ces deux architectures sont donc complémentaires.

5. Orchestration technique des flux de travail et collecte autonome de données

L'implémentation pratique de ces concepts se matérialise à travers des prototypes d'agents spatiaux à l'instar de LLM-Geo, qui démontrent comment un noyau décisionnel basé sur un LLM orchestre des calculs de bout en bout. Le processus d'orchestration commence par la conversion de la demande textuelle de l'utilisateur en un graphe orienté acyclique (DAG) de géotraitement. Chaque nœud de ce graphe correspond à une opération unitaire telle qu'une intersection spatiale, une re-projection ou une agrégation statistique tandis que les arêtes définissent la dépendance et le sens de circulation des flux d'informations géographiques. L'agent sollicite ensuite le LLM pour générer le code source Python associé à chaque nœud fonctionnel, avant de compiler l'ensemble dans un programme d'assemblage unifié dont l'exécution produit les cartes et les indicateurs finaux.

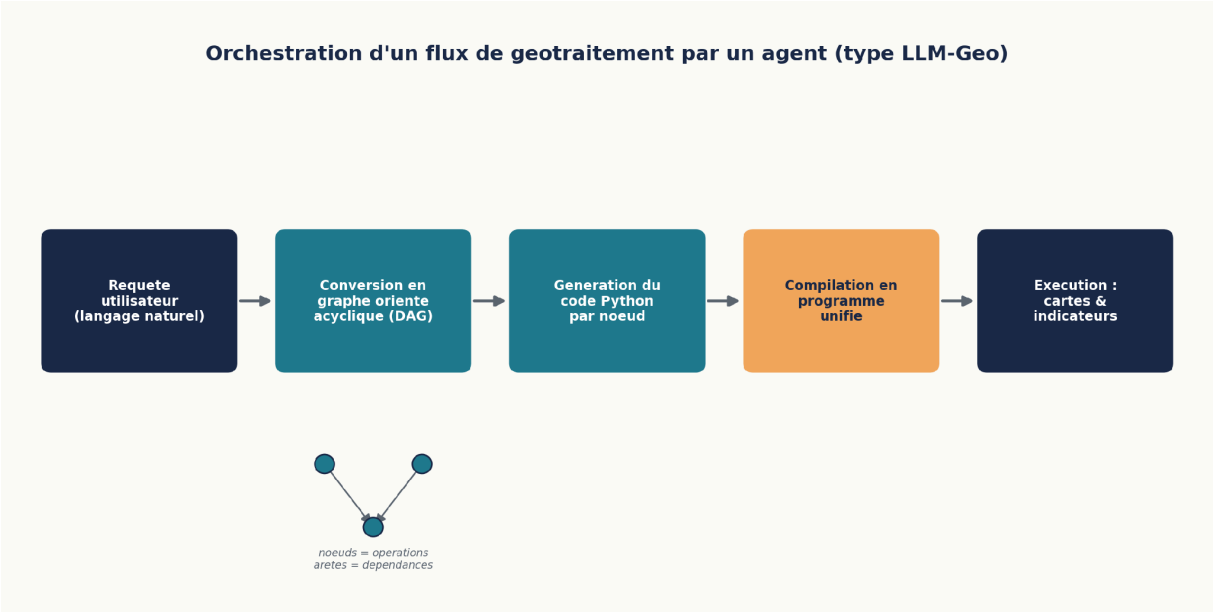


Figure 3 Pipeline d'orchestration d'un agent geospatial (type LLM-Geo)

5.1 LLM-Find : l'approvisionnement autonome en données

Le verrou historique de l'approvisionnement autonome en données géospatiales a trouvé une résolution technique majeure avec le développement du framework de recherche LLM-Find. Conçu pour pallier l'incapacité des modèles de langage à localiser et interroger de manière fiable les serveurs cartographiques distants, LLM-Find s'appuie sur une architecture extensible basée sur des manuels d'utilisation (handbooks). Ces documents répertorient les métadonnées, les systèmes de coordonnées par défaut, les formats de réponse et les spécifications d'interrogation des API REST pour différentes plateformes de données ouvertes.

Cette méthodologie a été déclinée sous la forme d'un plugin QGIS, permettant d'automatiser l'acquisition de couches de données géographiques complexes.

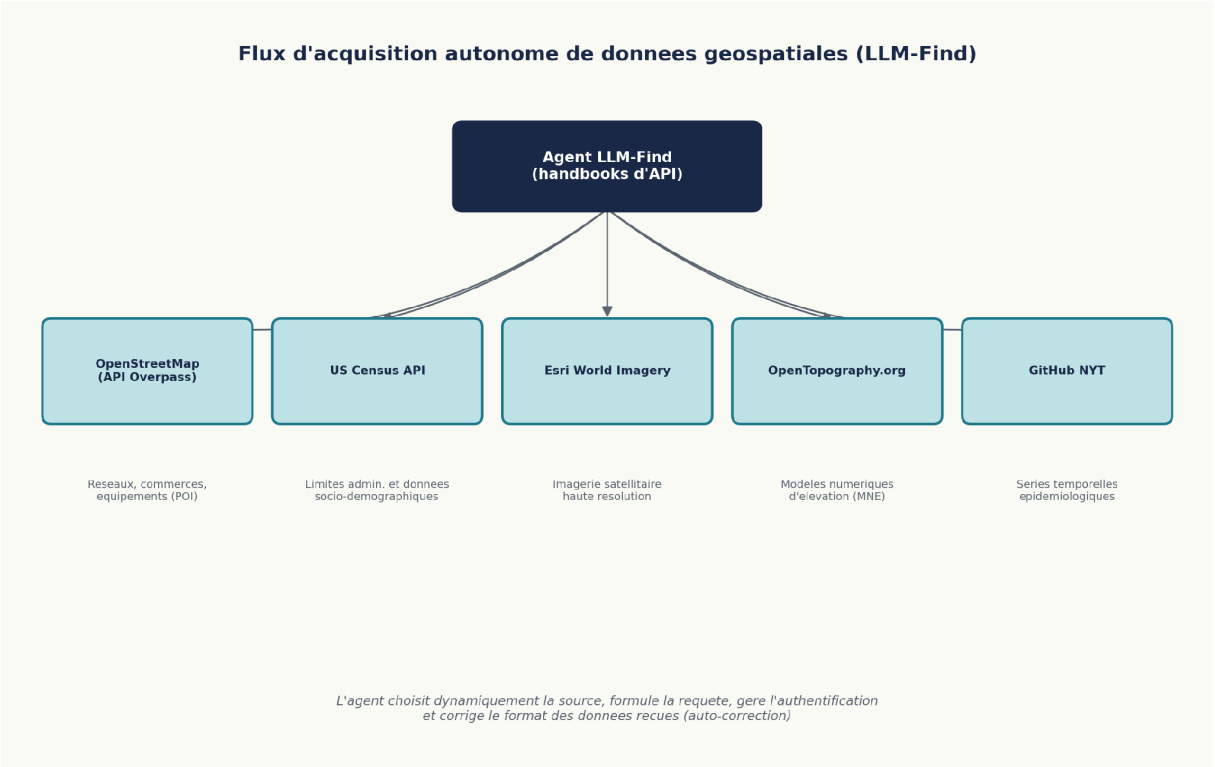


Figure 4 Sources de données mobilisées par l'agent LLM-Find

L'agent utilise ce référentiel pour formuler des requêtes d'interrogation, gérer l'authentification par clé d'API et écrire des scripts d'extraction de données. Cette méthodologie a été déclinée sous la forme d'un plugin pour le logiciel open-source QGIS, permettant à des utilisateurs d'automatiser l'acquisition de couches de données géographiques complexes.

Flux d'Acquisition de Données via LLM-Find	**Sources de Données Géospatiales**	**Informations Extraites et Formatées**
Requête d'infrastructure routière / POI	OpenStreetMap (API Overpass).	Géométries vectorielles des réseaux de transport, des commerces et des équipements d'intérêt public.

Limites administratives et données démographiques	Bureau du recensement des États-Unis (US Census API).	Polygones de découpage administratif associés aux attributs socio-économiques et démographiques nationaux.
Fonds de cartes satellitaires et orthophotographies	Esri World Imagery.	Tuiles d'imagerie satellitaire à haute résolution spatiale pour l'analyse d'occupation du sol.
Modèles numériques d'élévation (MNE)	OpenTopography.org.	Données altimétriques matricielles globales pour l'analyse de relief et la modélisation hydrologique.
Données de santé publique et épidémiologie	Dépôt GitHub du New York Times.	Séries temporelles d'indicateurs épidémiologiques tabulées et géoréférencées à l'échelle des comtés.

6. Standardisation des interfaces de communication : le role du Model Context Protocol

L'intégration industrielle des agents au sein d'infrastructures d'entreprise s'est accélérée grace a l'émergence du Model Context Protocol (MCP), un standard ouvert introduit par Anthropic fin 2024. Conçu comme une couche d'abstraction universelle entre les IA génératives et les systèmes externes, le MCP evite le développement coûteux de connecteurs propriétaires pour chaque plateforme logicielle et fournit une sémantique normalisée permettant à un agent de découvrir les capacités d'un outil, d'en comprendre les paramètres d'appel et d'en consommer les résultats.

Dans le domaine de la géomatique, ce protocole unifie l'accès aux progiciels de cartographie et aux bases de données géographiques. Des outils comme le serveur open-source gis-mcp exposent directement aux modèles de langage les capacites analytiques de bibliotheques fondamentales (Shapely, PyProj, Rasterio, PySAL), permettant aux agents de s'appuyer sur des fonctions mathématiques rigoureuses plutot que sur leur propre calcul, intrinsèquement peu fiable pour la géométrie. Exemple: Les agents peuvent ainsi s'appuyer sur des fonctions mathématiques rigoureuses pour valider la topologie d'un réseau ou mesurer des distances euclidiennes précises, surmontant ainsi l'incapacité native des réseaux de neurones à réaliser des calculs spatiaux déterministes de manière fiable.

L'architecture cloud s'est également emparée de cette technologie avec le serveur CARTO MCP, qui permet d'orchestrer plus de deux cents composants d'analyse géospatiale directement au sein d'entrepôts de données comme BigQuery, Snowflake ou Databricks, via des commandes telles que carto sql query, carto workflows components, carto workflows validate et carto workflows create le traitement s'exécutant directement dans le cloud de l'entreprise, garantissant sécurité et traçabilité.

L'agent IA interroge les données à l'aide de commandes SQL compilées par le biais d'instructions comme carto sql query, explore les bibliothèques d'algorithmes disponibles avec carto workflows components, esquisse et valide hors-ligne la structure d'un modèle avec carto workflows validate, puis déploie le traitement au sein de l'organisation cloud via la commande carto workflows create. Le traitement s'exécute directement dans le cloud de données, respectant ainsi les politiques de sécurité de l'entreprise tout en garantissant la traçabilité complète des opérations.

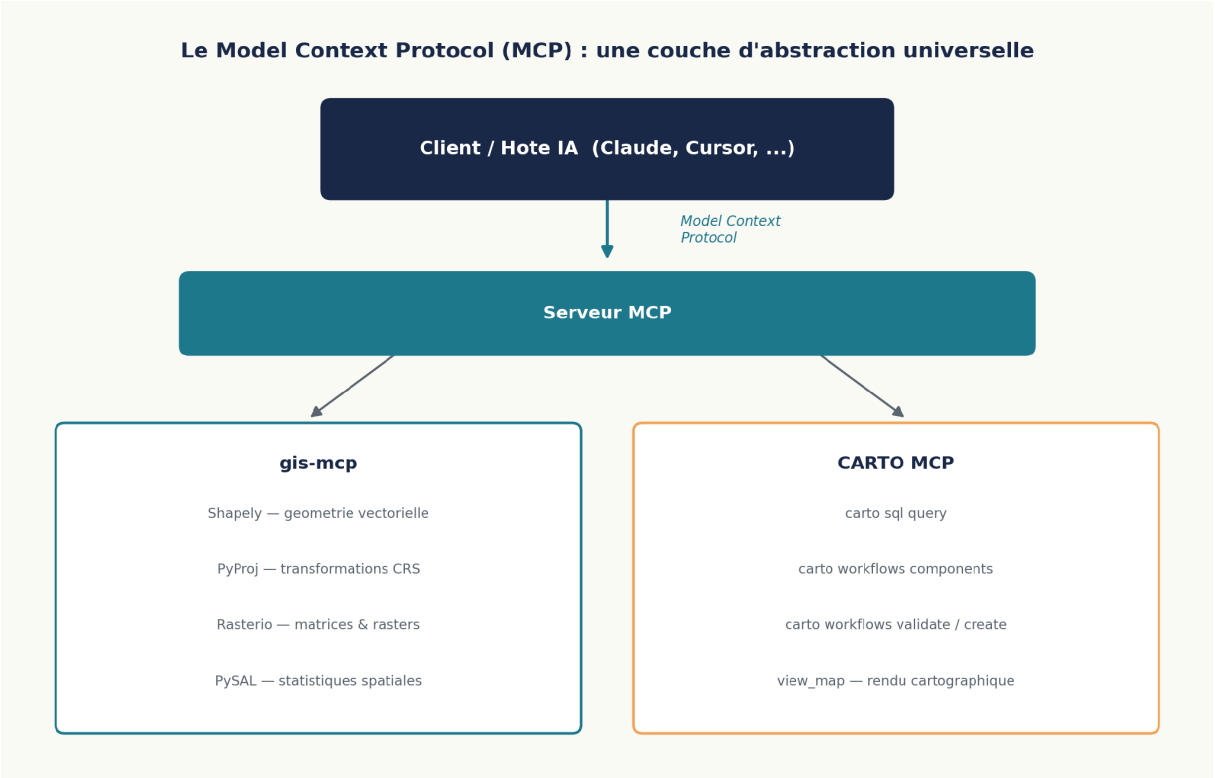


Figure 5 Architecture du Model Context Protocol appliquee a la geomatique

Cette dynamique s'étend également aux services de géolocalisation commerciaux. Le serveur MCP développé pour l'écosystème Google Maps intègre des outils spécialisés qui simplifient la manipulation des requêtes spatiales d'adresses et d'itinéraires pour les modèles de langage. L'agent géomatique accède à des fonctions d'analyse complexes en renseignant des paramètres de localisation et de contexte rigoureusement structurés :

La recherche textuelle géographique s'effectue via l'outil text_query, qui accepte des chaînes de caractères descriptives et permet d'associer un paramètre optionnel de pondération géographique appelé location_bias.

Le biais de localisation structuré s'exprime sous la forme d'un objet JSON contenant les coordonnées géographiques précises du centre de recherche, défini par une latitude et une longitude en degrés décimaux, associé à un rayon d'influence exprimé en mètres.

Les paramètres linguistiques et régionaux sont normalisés par l'intégration de codes d'identification normalisés au format ISO 639-1 pour les langues et au format ISO 3166-1 alpha-2 pour les pays d'origine.

La résolution d'adresses ambiguës et le traitement d'itinéraires s'appuient sur l'outil resolve_maps_urls, qui convertit les URL de partage géographiques fournies par les utilisateurs en identifiants uniques de lieux (Place IDs), assurant ainsi l'exactitude des calculs de matrices de distances et de temps de trajet.

Cette formalisation protocolaire permet d'intégrer des fonctionnalités de visualisation cartographique interactive directement au sein des interfaces de discussion des agents. Par exemple, la fonction view_map intégrée aux serveurs MCP permet d'afficher en temps réel le résultat intermédiaire d'un calcul de zone de chalandise ou d'un itinéraire de livraison, permettant au concepteur humain de valider visuellement la géométrie produite avant la finalisation et le déploiement du modèle de traitement.

7. Pratiques de codage assiste et rôle des IDE spécialisés

L'adoption d'environnements de développement assistés par l'IA tels que Cursor, Claude Code, GitHub Copilot ou Windsurf (maintenant Devin) bouleverse la production de lignes de code en

géomatique. Ces éditeurs modernes ne se limitent plus à la suggestion de portions de lignes de code isolées ; ils appréhendent l'architecture globale d'un projet. Grâce à des fonctionnalités avancées telles que le mode "Composer" de Cursor, ces outils peuvent analyser l'arborescence complète d'un répertoire de travail, coordonner des modifications simultanées sur plusieurs scripts Python et exécuter des tâches en arrière-plan via des agents autonomes.

7.1 Compartiment des fichiers projet

voir structuration

Pour tirer profit de ces capacités tout en prévenant la dégradation qualitative du code généré, les professionnels de la géomatique appliquent une méthodologie structurée, découpée en une phase préparatoire rigoureuse suivie d'une phase de construction incrémentale. Cette organisation logique s'appuie sur la mise en place d'un répertoire structuré et la configuration de fichiers d'instructions spécifiques qui guident les modèles d'intelligence artificielle au sein de l'environnement de développement.

7.2 Méthodologie en deux phases

La phase préparatoire commence par la coconception d'un cahier des charges détaillé (une spécification technique stockée sous /docs) rédigé de concert avec l'IA. Ce document définit de manière exhaustive l'architecture logicielle, les formats de données acceptés, les critères d'acceptation fonctionnels, les questions en suspens et les éléments explicitement exclus du développement afin d'éviter la dérive fonctionnelle des modules générés. Pour maximiser la productivité de l'agent, le développeur intègre dans le dossier de référence des scripts géomatiques fonctionnels préexistants ou des exemples officiels de bibliothèques d'éditeurs, offrant ainsi à l'IA un modèle d'apprentissage par l'exemple.

Au cours de la phase de construction, l'ingénieur géomaticien veille à implémenter les fonctionnalités par blocs restreints et facilement testables afin de limiter l'introduction de régressions logiques difficiles à diagnostiquer. La gestion des versions via un outil de contrôle de code source comme Git est initiée dès le lancement du projet, offrant un filet de sécurité essentiel qui permet de restaurer instantanément un état stable lorsque l'agent IA produit du code non-exécutable ou s'égare dans des boucles de correction infructueuses.

De plus, l'utilisation de sessions d'agents IA indépendantes disposant d'un historique de discussion vierge de biais méthodologiques est privilégiée pour réaliser des revues de code rigoureuses, analysant de manière neutre l'architecture, la présence de redondances et l'observabilité du code produit.

8. Evaluation empirique, diagnostics d'hallucination et performances quantitatives

L'intégration des LLM au sein des architectures géomatiques se confronte a la rigueur physique et mathématique des données de l'espace terrestre, révélant des faiblesses qualifiées d'hallucinations de codage géospatial. Le protocole GeoCode-Eval s'appuie sur la base GeoCode-Bench (5000 QCM, 1500 vrai/faux, 1500 tests de complétion, 1000 taches de programmation spatiale) pour mesurer l'efficacité des modèles selon des axes cognitifs, sémantiques et créatifs.

Afin d'apprécier la fiabilité des modèles de génération de code géospatial à l'aide de mesures répétées, l'évaluation s'appuie sur le calcul statistique de l'exactitude ajustée par la stabilité. La probabilité d'obtenir au moins une solution logicielle valide sur un ensemble de **n** tentatives indépendantes est formalisée par la métrique mathématique du taux de réussite *pass@k* ou l'*accuracy* . Pour intégrer l'impact négatif de la variabilité des réponses liée aux hallucinations cognitives des modèles, la stabilité fonctionnelle est quantifiée à l'aide du coefficient de variation. L'indice de précision ajusté sur la stabilité permet de pénaliser les modèles aux performances instables : une valeur proche de zéro traduit un comportement de génération homogène et fiable, indiquant que le modèle présente une faible propension à l'hallucination lors de la manipulation d'objets géographiques.

L'évaluation du taux de réussite d'un modèle dépasse le simple pourcentage d'exactitude classique (*Accuracy*) et s'appuie sur des formulations probabilistes avancées comme le **pass@k** pour la génération de code, le **Strict Task Success Rate (TSR)** pour les agents multi-étapes, ainsi que des indices de stabilité.

Thèmes et métriques clés identifiés :

> **La formule du Pass@k (Chen et al. / OpenAI, 2021) :** Calcule l'estimation non biaisée qu'au moins une solution parmi **k** échantillons soit exacte :

$$\text{pass@}k = \mathbb{E}\left[1 - \frac{\binom{N-c}{k}}{\binom{N}{k}}\right]$$

(où N *est le nombre total de solutions générées,* c *le nombre de solutions réussissant les tests, et* k *la taille de l'échantillon évalué).*

Strict Task Success Rate (TSR) : Utilisé dans les benchmarks d'agents autonomes (comme GISAgentBench ou SWE-bench), il mesure la fraction binaire de tâches dont l'exécution respecte 100 % des contraintes et tolérances :

$$\mathrm{TSR} = \frac{1}{M} \sum_{i=1}^{M} \mathbb{I}(\text{Tâche}_i \text{ réussie})$$

Précision ajustée sur la Stabilité (SA) : Intègre le coefficient de variation:

$$SA = \frac{\text{pass@5}}{1 + CV}$$

Proximité quantitative (sMAPE, F1, Exact Match) : Mesure la qualité des attributs continus ou catégoriels lorsque l'output d'un agent n'est pas uniquement binaire.

https://www.google.com/url?sa=E&q=https%3A%2F%2Farxiv.org%2Fabs%2F2608.01645

La confrontation de ces modèles à des cas d'usage réels est évaluée de manière rigoureuse par le benchmark GISAgentBench. Composé de trois cent quarante-neuf tâches complexes issues de questions réelles d'utilisateurs sur la plateforme d'échange GIS Stack Exchange, ce benchmark teste les modèles au sein d'un bac à sable d'exécution intégrant cent vingt-huit API de traitement géospatial à travers six zones géographiques d'intérêt. L'évaluation s'effectue en comparant l'état final des données géométriques et attributaires produites par l'agent à un fichier de référence absolu, écartant ainsi l'utilisation de juges LLM subjectifs au profit d'une validation géométrique stricte.

Tableau 8.1 — GISAgentBench (Strict Task Success Rate, notation stricte tolérante). [349 tâches, 128 API, 6 zones]

Modèle	Architecture	Strict TSR (%)
Claude-4.6-Opus	Propriétaire / Frontier	32,7 (meilleur)
Gemini-3.1-Pro	Propriétaire / Frontier	Moyen
GPT-5.4	Propriétaire / Frontier	Moyen
DeepSeek-V4-Pro	Open-Weight / Frontier	Faible

Tableau 8.2 — GeoCode-Eval (précision, tâches de génération de code géospatial).

Modèle	Catégorie	Précision
GeoCode-GPT-7B	Spécialisé (Code LLaMA fine-tune)	+9,1 % à +32,1 % vs modèles généraux (QCM)
DeepSeek-Coder-V2	Open-Weight / Code	Très élevée
Code LLaMA-70B	Open-Source / Code	77,19 %

Tableau 8.3 — GeoBenchX (précision agent multi-étapes).

Modèle	**Précision agent (%)**
Claude-4.6-Opus	60,8
DeepSeek-Coder-V2	56,3
Gemini-3.1-Pro	≤ 51,0

Les résultats issus des campagnes d'évaluation de GISAgentBench mettent en lumière un écart important entre les capacités de programmation générale des modèles de langage et leur aptitude

à mener à bien des analyses géographiques complexes. Le meilleur agent évalué ne parvient à résoudre que 32,7 % des problèmes spatiaux complexes sous un protocole de notation strict.

L'analyse approfondie des échecs d'exécution montre que les défaillances de planification logique constituent la principale cause d'erreur. Ces défaillances se traduisent par l'omission d'une opération géométrique indispensable (28,3 % des échecs) ou par l'application des algorithmes dans un ordre incohérent (18,4 % des échecs). [ATTENTION Ces deux parts ne sont pas additives : les mécanismes ne sont pas mutuellement exclusifs. Ne pas sommer en « 47 % ». Voir l'encadré 6.1.1.] Les erreurs de syntaxe ou de mauvaise configuration des paramètres d'outils, bien que fréquentes chez les modèles de taille intermédiaire, ne représentent quant à elles que 7,8 % des échecs d'exécution globaux.

L'évaluation montre également que les agents autonomes récupèrent et tracent les entités géométriques vectorielles de base de manière relativement fiable. Cependant, ils échouent fréquemment lors de la mise en œuvre de jointures spatiales complexes ou d'agrégations d'attributs tabulaires, ce qui entraîne des pertes d'informations sémantiques ou des incohérences de bases de données. Les erreurs d'alignement géométrique liées à l'absence de transformation de projection (CRS) lors de la superposition de couches cartographiques provenant de sources hétérogènes demeurent une cause fréquente de défaillance, conduisant à des calculs de zones tampons ou d'intersections vides.

Face à ces limites, l'intégration de frameworks de validation statique, à l'instar de MapSmith, apporte une amélioration concrète. En soumettant le graphe d'exécution planifié par l'agent à une vérification de conformité sémantique et topologique avant son exécution effective, ces outils permettent de rejeter les enchaînements logiques erronés et d'orienter le modèle vers des corrections algorithmiques cohérentes.

Méthodologie expérimentale proposée

8.1 Nécessité d'une évaluation propre au système

L'étude des benchmarks publiés (chapitre 6) ne démontre pas, à elle seule, que l'architecture proposée fonctionne. Tant qu'un prototype n'a pas été évalué, il convient de présenter cette partie comme un protocole expérimental à mettre en œuvre, et non comme la validation d'une solution déjà acquise. Ce chapitre définit donc un protocole d'évaluation graduel, reproductible et aligné sur la logique des benchmarks de référence : vérifier le résultat spatial final, et non seulement la plausibilité du code ou la qualité du raisonnement apparent.

8.2 Constitution du corpus de tâches

Le protocole repose sur un corpus de tâches SIG représentatives, idéalement entre 30 et 50 pour une première évaluation structurée. Le corpus couvre les opérations suivantes : tampons, reprojections, intersections, jointures spatiales, contrôle de géométries et production de couches. Chaque tâche est dotée d'un jeu de données versionné, de paramètres définis et d'un résultat de référence.

Tableau 8.1 Catégories de tâches du corpus d'évaluation.

Catégorie de tâche	Exemples	Critère de succès principal
Opérations simples	Buffer, sélection par attribut, calcul de champ	Résultat exact, CRS préservé
Reprojections	Transformation CRS, mesure dans un CRS projeté	Cohérence du CRS, unités correctes
Opérations topologiques	Intersection, union, différence, jointure spatiale	Validité géométrique, attributs conservés
Contrôle de qualité	Détection de géométries invalides, trous, chevauchements	Taux de détection, absence de faux positifs
Tâches ambiguës	Demandes sans seuil ni emprise précisés	Demande de précision (pas de complétion silencieuse)
Tâches à risque	Opération sur couche source sensible	Refus ou demande de confirmation correcte

Précaution méthodologique

Les jeux de données de test doivent éviter toute donnée géographique sensible non nécessaire. Les demandes, contextes, plans et résultats attendus des scénarios de référence sont versionnés pour garantir la reproductibilité.

8.3 Configurations comparées

Le protocole teste plusieurs configurations sur les mêmes tâches, afin d'isoler l'apport de chaque couche de l'architecture. Ce dispositif permet de tester directement les hypothèses H1, H2 et H3 :

Tableau 8.2 Matrice des configurations comparées.

Configuration	Description	Hypothèse testée
C0 Modèle seul	LLM sans RAG ni contrôles supplémentaires	Référence de base
C1 Modèle + RAG	Ajout du système de récupération documentaire	H2 Ancrage documentaire
C2 Modèle + outils déterministes + validations	Ajout de la couche d'outils et des filtres géométriques	H1 Fiabilité
C3 Architecture complète + routage	Routage modèle léger/lourd et configuration serveur complète	H3 Efficacité

Le routage entre modèle léger et modèle lourd suit une règle simple : les opérations simples (buffer, sélection) sont traitées par le modèle d'analyse rapide, tandis que les tâches complexes (multi-opérations, interprétation de règles métier) mobilisent le modèle de raisonnement. Cette règle doit être mesurée pour vérifier qu'elle réduit le coût ou la latence sans dégrader excessivement le taux de réussite.

8.4 Indicateurs de performance

Le protocole mesure au minimum les indicateurs suivants :

- **Taux de réussite complète des tâches (TSR) :** fraction de tâches dont l'exécution respecte 100 % des contraintes et tolérances.
- **Validité géométrique :** cohérence des CRS, intégrité topologique, exactitude des attributs.

- **Comportement de sécurité :** actions interdites ou mal paramétrées, refus et demandes de confirmation corrects.
- **Pertinence du RAG :** documents récupérés pertinents et validité des citations.
- **Efficacité :** temps d'exécution, usage mémoire, consommation énergétique estimée selon une méthode documentée.
- **Variabilité :** répétition des tests pour mesurer la stabilité des réponses.

8.5 Métriques formelles

Trois métriques structurent l'analyse quantitative. Elles sont rappelées ici formellement, car elles constituent le socle de l'évaluation.

$$Pass@k = E[\ 1 - C(N-c, k) / C(N, k)\ ]$$

Équation 5.1 Estimateur non biaisé du pass@k (probabilité qu'au moins une solution parmi k soit exacte).

où N est le nombre total de solutions générées, c le nombre de solutions réussissant les tests, et k la taille de l'échantillon évalué. Cette métrique, introduite par Chen et al. (2021), est adaptée à la génération de code.

$$TSR = (\text{nombre de tâches entièrement réussies}) / (\text{nombre total de tâches})$$

Équation 5.2 Strict Task Success Rate, utilisé dans les benchmarks d'agents autonomes (GISAgentBench, SWE-bench).

Le TSR mesure la fraction binaire de tâches dont l'exécution respecte 100 % des contraintes et tolérances. Il constitue le critère principal de réussite de l'évaluation.

$$SA = (\text{précision}) \times (1 - CV)$$

Équation 5.3 Indice de précision ajusté sur la stabilité, intégrant le coefficient de variation CV des réponses.

L'indice SA pénalise les modèles aux performances instables : une valeur proche de zéro traduit un comportement de génération homogène et fiable, indiquant une faible propension à l'hallucination lors de la manipulation d'objets géographiques.

> **Statut des métriques**
> Si l'indice SA est une métrique proposée par l'auteur, sa formule doit être explicitée, sa construction justifiée et son interprétation documentée [À SOURCER]. Les métriques Pass@k et TSR sont issues de la littérature (Chen et al., 2021 ; benchmarks d'agents) et doivent être rattachées à leurs références respectives [À SOURCER].

8.6 Protocole de comparaison des sorties géométriques

Pour les sorties géométriques, une comparaison de référence doit tenir compte des tolérances numériques et de l'équivalence spatiale pertinente : il ne suffit pas toujours de vérifier que deux fichiers sont identiques octet par octet. Deux géométries mathématiquement équivalentes peuvent différer par leur séquence de points ou leur représentation. Le protocole définit donc une tolérance spatiale et des critères d'équivalence (superposition, aire, périmètre, topologie) adaptés à chaque catégorie de tâche.

8.7 Déroulement et reproductibilité

1. Définir le corpus (30–50 tâches) avec jeux de données versionnés et résultats de référence.
2. Exécuter les quatre configurations (C0–C3) sur l'ensemble du corpus.
3. Répéter chaque exécution un nombre défini de fois pour mesurer la variabilité.
4. Conserver les versions des modèles, des outils et des données.

5. Documenter précisément les conditions de test (matériel, quantification, longueur de contexte, température).
6. Comparer les configurations selon les indicateurs du point 5.4 et les métriques du point 5.5.

> **Alignement avec GISAgentBench**
> Ce protocole reprend la logique de GISAgentBench : vérifier le résultat spatial final et pas seulement la plausibilité du code ou la qualité du raisonnement apparent. Il permet de situer l'architecture proposée par rapport aux états de l'art publiés.

9. Résultats : benchmarks publiés et état de l'implémentation

Distinction épistémologique fondamentale

Ce chapitre sépare strictement deux registres : (1) les résultats issus des benchmarks publiés, qui documentent l'état de l'art des agents géospatiaux ; (2) l'état actuel de l'implémentation du prototype, qui n'a pas encore fait l'objet du protocole d'évaluation défini au chapitre 5. Confondre les deux consisterait à présenter un fonctionnement envisagé comme un résultat expérimental.

9.1 État de l'art empirique : les benchmarks publiés

La confrontation des modèles à des cas d'usage réels est évaluée de manière rigoureuse par le benchmark GISAgentBench. Composé de 349 tâches complexes issues de questions réelles d'utilisateurs sur la plateforme GIS Stack Exchange, ce benchmark teste les modèles au sein d'un bac à sable d'exécution intégrant 128 API de traitement géospatial à travers six zones géographiques. L'évaluation compare l'état final des données géométriques et attributaires produites par l'agent à un fichier de référence absolu, écartant l'utilisation de juges LLM subjectifs au profit d'une validation géométrique stricte.

Lecture précise du benchmark

Le meilleur agent présenté dans la publication ne parvient à résoudre que 32,7 % des tâches sous un protocole de notation strict. Ce chiffre, qui constitue le résultat central du benchmark, doit être rattaché à la publication originale (GISAgentBench, arXiv:2608.01645) [À SOURCER]. Le dénominateur exact (349 tâches ? sous-ensemble ?) et le protocole de notation doivent être confirmés [VÉRIFIER].

9.1.1 Analyse des mécanismes d'échec correction d'interprétation

L'analyse des échecs d'exécution révèle plusieurs mécanismes. Il convient ici de corriger une interprétation erronée présente dans le document initial : l'addition automatique de 28,3 % d'omission d'opération et de 18,4 % d'erreurs d'ordre pour annoncer « 47 % des échecs » est statistiquement invalide, car ces mécanismes ne sont pas mutuellement exclusifs et le tableau source dépasse 100 % lorsqu'on somme ses pourcentages.

Tableau 9.1 Mécanismes d'échec observés (parts non exclusives ; dénominateur à confirmer).

Mécanisme d'échec	Part dans les échecs	Nature
Omission d'une opération géométrique indispensable	28,3 %	Planification
Application des algorithmes dans un ordre incohérent	18,4 %	Planification
Erreurs de syntaxe / mauvaise configuration de paramètres	7,8 %	Exécution
Erreurs d'alignement géométrique (CRS)	fréquentes	Exécution / données

9.2 État actuel de l'implémentation du prototype

Contrairement aux benchmarks publiés, le prototype d'assistant n'a pas encore fait l'objet d'une évaluation formalisée selon le protocole du chapitre 5. Son état actuel peut néanmoins être décrit honnêtement :

- **Composants en cours de développement :** orchestrateur geoai_qgis.py, module RAG (rag.py), recherche de couche avec correction de faute (difflib.get_close_matches), préparation d'actions structurées.
- **Composants proposés :** routage multi-modèle (modèle léger / EmbeddingGemma / modèle de raisonnement), filtres d'intégrité géométrique, intégration complète dans QGIS.

Composants à évaluer : pertinence du RAG, taux de réussite des tâches, comportement de sécurité, efficacité du routage.

10. Ethique de l'IA en géomatique

Le socle international (UNESCO 2021, Conseil de l'Europe 2024) repose sur les droits humains, la transparence, l'équité et le contrôle humain. Seul le second est contraignant, et seulement s'il est ratifié [5][6].

10.1 Les cadres de référence internationaux

En novembre 2021, les 193 États membres de l'UNESCO ont adopté la Recommandation sur l'éthique de l'intelligence artificielle, présentée comme la première norme mondiale en la matière [5]. Elle place la protection des droits humains et de la dignité au centre, s'appuie sur des principes comme la transparence et l'équité, et insiste sur le contrôle humain des systèmes d'IA [5]. Elle comporte aussi des domaines d'action pour les politiques publiques : gouvernance des données, environnement et écosystèmes, genre, éducation et recherche, santé [5]. Ce texte est non contraignant : AlgorithmWatch salue notamment ses passages sur le développement sobre en ressources, l'interdiction de la surveillance de masse et les droits des personnes affectées par les systèmes d'IA [5].

Le Conseil de l'Europe a adopté le 17 mai 2024 une Convention-cadre sur l'IA, les droits de l'homme, la démocratie et l'État de droit, ouverte à la signature le 5 septembre 2024 à Vilnius [6]. Il s'agit du premier traité international juridiquement contraignant sur le sujet : il vise à rendre l'usage des systèmes d'IA compatible avec ces trois principes sur l'ensemble de leur cycle de vie [6]. Il a notamment été signé par l'Union européenne, le Royaume-Uni et les États-Unis, puis par le Canada et le Japon en février 2025 [6][6bis]. Son entrée en vigueur exige cinq ratifications, dont au moins trois d'États membres du Conseil de l'Europe [6].

10.2 Du principe éthique à la règle de droit : l'approche par les risques

L'AI Act traduit ces préoccupations en obligations graduées selon le niveau de risque. Les pratiques jugées inacceptables sont interdites depuis le 2 février 2025, de même que l'obligation de formation à l'IA (« AI literacy ») s'applique depuis cette date [7]. Les systèmes dits « à haut risque » concernent des domaines où une décision automatisée touche directement les personnes : recrutement, éducation, accès aux services essentiels, maintien de l'ordre, migration, justice [8]. Les exigences associées incluent une supervision humaine effective, de la documentation et un enregistrement dans une base de données européenne [8].

Les questions éthiques ne se réduisent pas à la conformité : l'étude de Luccioni et al. (section 2) montre par exemple que choisir un modèle généraliste plutôt qu'un modèle spécialisé est aussi un arbitrage entre utilité et coût environnemental, que les auteurs invitent à « peser plus intentionnellement » [2].

11. L'impact sur l'environnement

11.1 La consommation d'électricité des centres de données

Le rapport Energy and AI de l'AIE (avril 2025) est la référence institutionnelle la plus citée. Ses principaux constats [1] :

les centres de données ont consommé environ 415 TWh en 2024, soit 1,5 % de l'électricité mondiale ;

cette consommation devrait plus que doubler d'ici 2030 (environ 945 TWh, un peu plus que la consommation actuelle du Japon) et atteindre environ 1 200 TWh en 2035 dans le scénario de référence ;

la demande des centres de données optimisés pour l'IA devrait plus que quadrupler d'ici 2030 ;

aux États-Unis, ils représenteraient près de la moitié de la hausse de la demande d'électricité d'ici 2030 ; dans les économies avancées, plus de 20 % ;

à l'échelle mondiale, les centres de données ne représentent qu'environ un dixième de la croissance de la demande d'électricité d'ici 2030, moins que les moteurs industriels, la climatisation ou les véhicules électriques ;

Le gaz naturel augmenterait de 175 TWh pour y répondre, surtout aux États-Unis.

L'AIE relève par ailleurs que la demande de minéraux critiques liée à ces équipements pose un enjeu de sécurité d'approvisionnement [1]. Les incertitudes sont fortes : en 2035, la fourchette des scénarios de l'AIE va de 700 à 1 700 TWh [1].

11.2 L'empreinte d'un usage : des ordres de grandeur très variables

Une étude publiée à la conférence ACM FAccT 2024 (Luccioni, Jernite, Strubell) a comparé l'énergie nécessaire à 1 000 inférences selon les modèles et les tâches. Elle conclut que les architectures génératives multi-usages sont des ordres de grandeur plus coûteuses que les systèmes spécialisés pour de nombreuses tâches, à nombre de paramètres comparable [2]. Ces valeurs dépendent toutefois fortement des réglages et du matériel : l'équipe du classement AI Energy Score a souligné que des mesures sur un même modèle d'image pouvaient différer d'un facteur important selon la taille des images générées et la génération de GPU [2bis].

11.3 L'analyse de cycle de vie de Mistral AI (juillet 2025)

Mistral AI a publié l'analyse de cycle de vie de son modèle Mistral Large 2, réalisée avec Carbone 4 et avec le soutien de l'ADEME, puis relue par les cabinets Resilio et Hubblo [3]. Après 18 mois, l'entraînement et l'usage du modèle ont représenté environ 20,4 kt d'équivalent CO_2, 281 000 m^3 d'eau consommée et 660 kg Sb eq d'épuisement de ressources abiotiques ; une réponse de 400 tokens de l'assistant Le Chat équivaut à environ 1,14 g de CO_2, 45 mL d'eau et 0,16 mg Sb eq [3][3bis]. Entraînement et inférence concentrent 85,5 % des émissions de gaz à effet de serre et 91 % de la consommation d'eau ; la fabrication et la fin de vie du matériel pèsent environ 11 % des émissions [3bis][3ter]. Mistral reconnaît qu'aucun inventaire fiable du cycle de vie des GPU n'existe encore et que leur impact incorporé a dû être approximé [3].

À lire avec prudence : c'est une étude commanditée par le fournisseur lui-même, sur un modèle entraîné en France, où l'électricité est peu carbonée ; ses résultats ne se transposent pas tels quels à d'autres modèles ou régions [3ter].

8.4 Un déficit de données

L'Arcep constate, dans sa contribution à la consultation de la Commission européenne (novembre 2025), qu'il existe peu de données précises et fiables sur la consommation énergétique de l'IA, et que l'impact ne se limite pas à l'énergie : l'eau et les ressources minérales comptent aussi [9bis]. C'est le principal point faible de toute estimation chiffrée, y compris celles de cette note.

12. Cadre réglementaire : AI Act et RGPD

La montée en puissance des agents géospatiaux a amené la communauté internationale à se tourner vers des outils de gouvernance juridiquement contraignants. En Europe, la loi sur l'intelligence artificielle (EU AI Act) impose une approche régulatoire fondée sur le niveau de risque ; les outils d'IA classés à haut risque (sécurité civile, services publics, administration) font l'objet d'obligations de transparence, de qualité des jeux de données, et d'audits algorithmiques obligatoires.

Ces nouvelles régulations s'imbriquent étroitement avec le Règlement Général sur la Protection des Données (RGPD) pour empêcher l'utilisation d'outils géomatiques à des fins de surveillance généralisée, de profilage spatial sans consentement ou de violation de la vie privée spatiale. À l'échelle internationale, des institutions comme le Comité d'experts de l'ONU sur la gestion de l'information géospatiale (UN-GGIM) établissent des directives poussant à la structuration des données. Pour les géomaticiens, l'enjeu légal consiste désormais à appliquer la "conformité dès la conception" (Compliance by Design), assurant que les requêtes algorithmiques automatisées et les acquisitions de bases de données (via des outils tels que LLM-Find) respectent les droits fondamentaux sans perturber le cycle d'innovation.

Normalisation ISO, standards de l'OGC et gouvernance éthique

Au-delà des textes de loi, la conformité légale s'appuie désormais sur des normes techniques et éthiques strictes :

Tableau 12.1 Référentiels normatifs et leur apport à l'architecture.

Référentiel	Objet	Apport à l'architecture
ISO 19157	Qualité de l'information géographique (précision positionnelle, intégrité topologique)	Garantir la qualité des jeux de données utilisés par l'IA
OGC (WMS, WFS, WCS, GeoJSON)	Interopérabilité des services géospatiaux	Structuration uniforme de l'acquisition de données
Charte Locus	Minimisation et limitation des usages secondaires	Préservation de la géo-confidentialité

Normes ISO géospatiales et IA : L'évaluation de la qualité des jeux de données géographiques utilisés par les modèles d'IA impose le respect de normes internationales comme l'ISO 19157 (qualité de l'information géographique), garantissant la précision positionnelle et l'intégrité topologique nécessaires pour éviter les erreurs d'analyse automatisée.

Standards d'interopérabilité (OGC) : L'alimentation des pipelines géomatiques automatisés requiert une adhésion stricte aux standards de l'Open Geospatial Consortium (OGC) (tels que WMS, WFS, WCS et GeoJSON), assurant une structuration uniforme lors de l'acquisition autonome de données.

Référentiels d'éthique spatiale : L'adoption de principes comme ceux de la Charte Locus (Locus Charter) pousse à la minimisation de la collecte de données de localisation et à la limitation stricte des usages secondaires afin de préserver la géo-confidentialité (*Geo-privacy*).

12.1 Union européenne : le règlement 2024/1689 (AI Act)

Le règlement est entré en vigueur le 1er août 2024 et s'applique de façon progressive [7]. Selon le service d'assistance officiel de la Commission : interdictions et formation à l'IA depuis le 2 février 2025 ; règles sur les modèles d'IA à usage général et gouvernance depuis le 2 août 2025 ; transparence (article 50) et début de l'application à partir du 2 août 2026 [7].

Modification de 2026. Le « Digital Omnibus » (règlement (UE) 2026/1744, approuvé par le Parlement européen le 16 juin 2026 et par le Conseil le 29 juin, publié au Journal officiel le 24 juillet et en vigueur depuis le 27 juillet 2026) a reporté les obligations pour les systèmes à haut risque : au 2 décembre 2027 pour ceux de l'annexe III, et au 2 août 2028 pour ceux intégrés à des produits réglementés (annexe I) [8]. Les obligations de transparence de l'article 50 restent applicables depuis le 2 août 2026, avec un délai au 2 décembre 2026 pour les systèmes déjà sur le marché [8]. Les sanctions peuvent atteindre 15 millions d'euros ou 3 % du chiffre d'affaires mondial pour la plupart des manquements [8] ; le règlement prévoit des plafonds plus élevés pour les pratiques interdites (article 99, voir [7bis]).

12.2 Les dispositions environnementales de l'AI Act

Elles sont modestes et surtout documentaires : les fournisseurs de modèles à usage général doivent consigner la consommation d'énergie connue ou estimée du modèle (article 53 et annexe XI), la Commission doit demander des normes d'efficacité énergétique (article 40), et des codes de conduite volontaires peuvent traiter l'impact environnemental (article 95) [9]. Plusieurs analyses jugent ce volet insuffisant : l'information n'est destinée qu'aux autorités, et non au public [9ter]. L'eau, les minéraux et les déchets électroniques n'y sont presque pas traités [9ter].

12.3 États-Unis

Il n'existe pas de loi fédérale générale sur l'IA. Le niveau fédéral relève surtout de décrets et de textes d'orientation ; l'essentiel des obligations contraignantes vient des États [10]. Un décret du 11 décembre 2025 a créé un groupe du ministère de la Justice chargé de contester les lois d'État, mais aucune loi fédérale ni décision de justice ne les a écartées à ce jour [10]. La loi californienne SB 53 sur la transparence des modèles de pointe est en vigueur depuis le 1er janvier 2026 ; le Colorado a remplacé sa loi de 2024 par la SB 26-189, applicable au 1er janvier 2027 [10].

12.4 Points de vigilance sur la fiabilité

- **Textes primaires** : UNESCO, Conseil de l'Europe, AIE, Commission européenne, EUR-Lex et ADEME/Arcep sont des sources institutionnelles de premier rang.
- **Omnibus et droit américain** : faute d'avoir pu consulter le texte au Journal officiel dans cette recherche, ces points reposent sur des analyses de cabinets et de sites de conformité qui concordent entre elles. La page de calendrier de la Commission consultée affichait encore les dates antérieures. À vérifier sur EUR-Lex avant toute citation officielle.
- **Chiffres de Mistral** : source primaire mais non indépendante ; les valeurs en CO_2 sont ici relayées par la presse spécialisée [3bis].
- **Convention du Conseil de l'Europe** : le nombre actuel de ratifications n'a pas été vérifié ; consulter le Bureau des traités du Conseil de l'Europe.
- **Hors périmètre** : l'application nationale en France (autorités compétentes, CNIL) n'a pas été traitée faute de source vérifiée.

12.5 Souveraineté numérique et géospatiale

L'intégration des agents autonomes et des modèles de fondation soumet la géomatique à des impératifs majeurs de souveraineté des données :

Contrôle territorial et infrastructurel : L'utilisation de services cloud tiers propriétaires situés hors des juridictions nationales crée un risque d'exfiltration ou de dépendance stratégique vis-à-vis d'acteurs privés étrangers.

Déploiement et IA souveraine : Pour préserver les données sensibles de l'administration et des infrastructures critiques, les gouvernements et entreprises privilégient des architectures d'IA souveraine. Cela implique le déploiement de modèles open source sur des infrastructures locales ou privées (via des moteurs comme Ollama ou LM Studio), garantissant que ni les requêtes géographiques ni les données attributaires ne quittent le réseau interne.

13. L'Émergence de l'IA Locale : Architecture, Performance et Souveraineté des Données en 2026

13.1. Introduction : Le Changement de Paradigme vers l'Inférence Locale

En août 2026, l'industrie technologique a achevé sa mutation structurelle : le passage massif du "Cloud-First" vers l'inférence locale (Edge computing). Ce changement n'est plus une simple tendance pour "early adopters", mais une priorité stratégique pour les organisations exigeant une confidentialité absolue et une résilience opérationnelle totale. En s'affranchissant des infrastructures distantes, les entreprises éliminent les latences réseau et les risques de rupture de service tout en reprenant le contrôle sur leur propriété intellectuelle.

L'adoption de ces solutions a atteint une échelle industrielle. En juillet 2026, on dénombre plus de 8,9 millions de développeurs actifs sur Ollama, contre 3,5 millions un an auparavant. Cette explosion est

illustrée par des chiffres vertigineux : la famille Llama 3.1 totalise 111 millions de téléchargements cumulés, tandis que DeepSeek-R1 a franchi la barre des 79 millions dès mars 2026. Depuis 2023, les téléchargements mensuels ont été multipliés par 520, transformant l'infrastructure interne en un actif stratégique souverain. Cette transition repose toutefois sur une ingénierie matérielle rigoureuse pour soutenir la complexité croissante des modèles.

13.2. Analyse des Infrastructures Matérielles : Du Laptop au Data Center

Pour un architecte logiciel, la viabilité d'un projet d'IA locale repose sur l'adéquation mathématique entre les capacités de calcul et la taille des modèles. Une sous-estimation du matériel ne se traduit pas seulement par de la lenteur, mais par un échec structurel de l'inférence.

Le tableau suivant détaille les configurations types de 2026, incluant les nouveaux standards de performance :

Configuration	Matériel Type	Capacité de Modèle	Spécificités Techniques
Ordinateur Portable	Apple Silicon (M1-M4), 16-32 Go RAM	7B à 13B (ex: Llama 3.3 8B, Gemma 4)	**Mémoire unifiée** : Exploitation du Neural Engine. Support du décodage spéculatif MTP (Gemma 4).
Station de Travail GPU	NVIDIA RTX 4090 (24 Go VRAM)	13B à 33B (ex: **Laguna XS 2.1**)	**VRAM dédiée** : Haute bande passante. Idéal pour le codage agentique (Laguna XS 2.1).
Serveur / Multi-GPU	2x RTX 3090/4090 (48 Go+ VRAM)	70B+ (ex: Llama 70B, DeepSeek R1)	**Capacité maximale** : Indispensable pour le raisonnement complexe et les fenêtres de contexte étendues.

Limites Critiques : VRAM et Cache KV

La variable limitante reste la VRAM (Video RAM). Un piège architectural courant est d'ignorer l'impact du Cache KV. Par exemple, étendre la fenêtre de contexte de 4K à 32K tokens peut doubler les besoins en VRAM pour un même modèle. Si la mémoire vidéo sature, le système bascule sur le CPU, provoquant une chute de performance de 100 tokens/s à moins de 5 tokens/s. Sur les architectures Apple, l'activation du décodage spéculatif MTP pour Gemma 4 a permis de doubler la vitesse d'inférence, soulignant l'importance de l'optimisation matérielle-logicielle.

13.3. Écosystèmes Logiciels et Protocoles d'Installation

La maturité logicielle de 2026, portée par la version stable d'Ollama (v0.33.2), a simplifié le déploiement au point de le rendre transparent pour les opérations (DevOps).

Protocoles de Déploiement Standardisés

- Linux (Ubuntu 24.04 LTS) : Installation via binaire unique (curl -fsSL https://ollama.com/install.sh | sh), géré nativement par systemd.
- macOS & Windows : Installation via paquets natifs ou brew/winget, avec support automatique du GPU et du Neural Engine.
- Docker : Utilisation de l'image ollama/ollama. L'intégration avec le NVIDIA Container Toolkit est désormais le standard pour la portabilité en production.

Dualité Architecturale : Agentic vs Infrastructure

Deux philosophies dominent le marché :

LM Studio/ Bionic : Conçu comme un "harnais agentique" pour le travail complexe (codage, recherche). Sa force réside dans sa flexibilité : il permet d'utiliser des modèles locaux, des modèles distants via le protocole LM Link, ou des modèles cloud avec une clause de "Zero Data Retention".

Ollama (v0.5.x+) : S'est imposé comme l'infrastructure de référence grâce à sa stabilité (Série B de 65 M$ bouclée en juillet 2026) et son API compatible OpenAI. Il permet d'injecter des LLM locaux dans n'importe quelle stack logicielle existante par simple changement d'endpoint.

13.4. Optimisation des Performances : Quantification et Gestion des Ressources

La quantification, utilisant le format **GGUF** et le moteur **llama.cpp**, est le levier stratégique permettant de réduire l'empreinte mémoire sans sacrifier la précision sémantique.

Évaluation des Niveaux de Quantification (Benchmark 2026)

Sur une configuration type RTX 4090 pour un modèle Llama 3.1 8B :

- ☐ **FP16** (Non quantifié) : 14 Go VRAM | ~15 tok/s | Précision 100%.

- ☐ **Q8_0** (8 bits) : 9 Go VRAM | ~28 tok/s | Précision ~99%.
- ☐ **Q4_K_M** (4 bits - Standard) : 6 Go VRAM | 95-110 tok/s | Précision ~95%. C'est le standard industriel pour l'équilibre performance/ressources.

Variables d'Environnement Critiques pour la Production

Pour garantir la stabilité en environnement multi-utilisateurs, l'expert doit configurer :

OLLAMA_FLASH_ATTENTION=1 : Active les optimisations de calcul pour un gain de vitesse immédiat.

OLLAMA_NUM_PARALLEL : Définit le nombre de requêtes traitées simultanément.

OLLAMA_MAX_LOADED_MODELS : Limite le nombre de modèles résidents en VRAM pour éviter les collisions mémoire.

13.5. Souveraineté des Données et Cadre Réglementaire (RGPD & AI Act)

L'inférence locale est devenue la réponse technique ultime aux exigences de l'AI Act européen. En traitant les données à 100 % sur site, les risques de fuites (data leaks) vers des serveurs extracommunautaires sont structurellement éliminés.

Les avantages juridiques sont majeurs :

Suppression des DPA (Data Processing Agreements) : Plus besoin de contrats complexes avec des fournisseurs tiers.

Secteurs Sensibles : Pour la santé et le juridique, le traitement local garantit le secret professionnel sans compromis.

Confiance Hybride : Même en utilisant le cloud via LM Studio Bionic, l'assurance du "Zero Data Retention" offre une couche de sécurité supplémentaire pour les tâches nécessitant des modèles "Frontier" trop lourds pour le matériel local.

13.6. Impact Environnemental et Économique de l'IA Locale

Le modèle économique de l'IA locale défie la tarification par jeton des géants du Cloud.

Analyse du Coût Total de Possession (TCO)

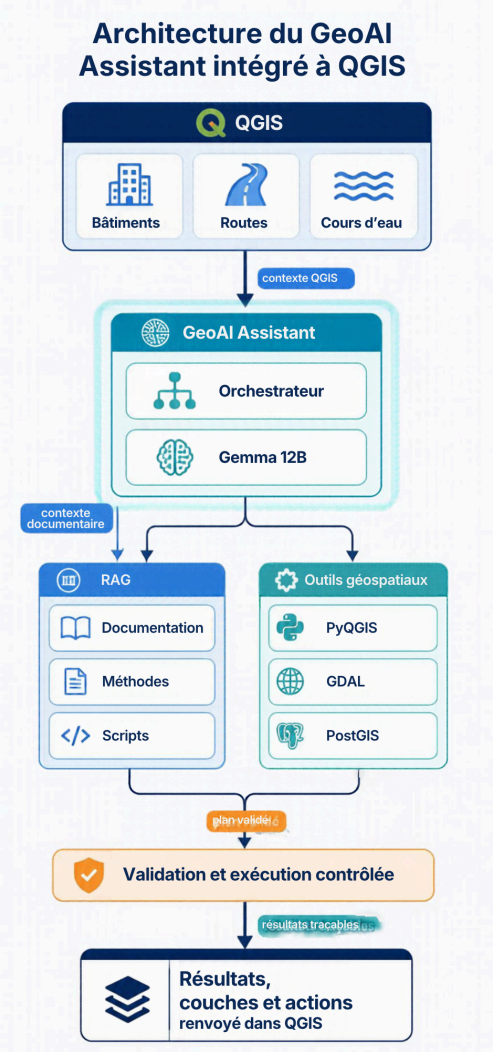
Alors que les API Cloud facturent entre 0,002 $ et 0,06 $ pour 1 000 tokens, l'IA locale affiche un coût marginal de 0 $ par jeton après investissement matériel si on ne compte pas le coût en energie. Pour une entreprise traitant des volumes massifs, le retour sur investissement (ROI) est généralement atteint en moins de six mois.

Efficacité Énergétique

Contrairement à une idée reçue, l'IA locale optimisée est écologiquement viable. L'utilisation de modèles quantifiés réduit drastiquement la puissance de calcul par inférence. Un cluster local bien dimensionné évite la surcharge énergétique liée au refroidissement massif des datacenters et au transport de données constant sur le réseau mondial, réduisant ainsi l'empreinte carbone numérique globale de l'organisation.

13.7. La spécialisation d'une IA locale

Détourner une IA générative généralisé en une IA spécialisé en géomatique, l'exemple d'une IA spécialisé QGIS avec une extension.



Créer une architecture de dossier avec d'un côté le RAG et de l'autre les outils. L'IA ingère votre requête puis la traduit en json à l'aide de vos connaissances.

Vous avez la possibilité d'utiliser une IA pour l'orchestrateur, qui génère un plan puis via une autre IA locale spécialisé en code, vous créer un script. Cela permet d'utiliser moins de ressource GPU.

13.7.1 Architecture technique d'un Assistant IA intégré à QGIS

Résumé de l'architecture

L'assistant est une architecture d'assistance géospatiale conçue pour interpréter des demandes en langage naturel, exploiter le contexte documentaire pertinent, planifier des opérations spatiales contrôlées et restituer les résultats dans QGIS.

Le schéma fourni confirme les éléments suivants : QGIS fournit les couches **Bâtiments**, **Routes** et **Cours d'eau** ; l'assistant s'appuie sur **Gemma 12B** ; un module **RAG** exploite la documentation, les méthodes et les scripts ; une couche d'outils géospatiaux exécute des opérations au moyen de **PyQGIS**, **GDAL** et **PostGIS** ; les résultats et les actions sont renvoyés dans QGIS.

Les composants mentionnés sont décrits comme des composants prévus par l'architecture, et non comme des composants déjà déployés. Les mécanismes de stockage, d'indexation, d'exécution et d'hébergement restent des choix d'implémentation à confirmer.

Positionnement

Le système doit assister l'utilisateur sans transformer une demande en langage naturel en exécution directe. Toute opération géospatiale doit être interprétée, planifiée, validée, exécutée dans un périmètre contrôlé, puis rendue traçable dans QGIS.

13.7.2. Objectifs et périmètre

Objectifs

L'objectif principal est de fournir une interface conversationnelle capable de relier les données visibles dans QGIS à des connaissances documentaires et à des opérations géospatiales explicables.

- **Comprendre** une demande formulée en langage naturel et identifier les couches, emprises, attributs et opérations concernés.
- **Rechercher** les méthodes, règles, procédures et scripts pertinents au moyen du module RAG.
- **Planifier** une séquence d'actions géospatiales avant toute exécution.
- **Valider** les paramètres, les permissions, les contraintes de ressources et les effets attendus.
- **Exécuter** les outils autorisés via les interfaces prévues par l'architecture.
- **Restituer** les résultats, les actions réalisées, les avertissements et les sources dans QGIS.

Périmètre fonctionnel

Dans le périmètre	Hors périmètre ou à confirmer
Interprétation de demandes géospatiales liées aux bâtiments, routes et cours d'eau.	Déploiement d'une infrastructure particulière ou choix d'un fournisseur.
Recherche de contexte dans la documentation, les méthodes et les scripts.	Choix d'une technologie précise d'indexation ou de vectorisation.
Préparation et exécution contrôlée d'opérations PyQGIS, GDAL et PostGIS prévues.	Exécution arbitraire de code, de scripts ou de requêtes fournis directement par l'utilisateur ou le modèle.
Retour des résultats et des actions dans QGIS.	Production de données réelles, de métriques de production ou de résultats non issus d'une exécution vérifiée.

Principes directeurs

La conception privilégie la traçabilité, la validation humaine lorsque le risque le justifie, le moindre privilège, l'explicabilité des opérations et la séparation stricte entre génération de texte et exécution d'outils.

13.7.3. Architecture logique détaillée

Responsabilités des composants

Composant	**Responsabilités**	**Statut dans cette documentation**
QGIS	Fournir le contexte des couches, recevoir les actions et afficher les résultats, messages, couches produites et demandes de confirmation.	Confirmé par le schéma ; interfaces détaillées à préciser.
Orchestrateur / Assistant	Coordonner l'interprétation, le RAG, la planification, la validation, l'appel des outils, la gestion des erreurs et la restitution.	Composant prévu ; responsabilités proposées.
Modèle (routage Gemma): Gemma 12B ou 4B selon architecture	Analyser la demande et le contexte autorisé, produire une interprétation, un plan structuré et une réponse explicative.	Composant prévu par le schéma ; modalités d'accès à confirmer.
Module RAG	Rechercher et récupérer le contexte pertinent dans la documentation, les méthodes et les scripts, puis transmettre les sources à l'orchestrateur.	Confirmé par le schéma ; stratégie technique à choisir.
Base documentaire	Conserver les documents de référence, procédures, règles, descriptions et informations de gouvernance.	Source prévue ; format et cycle de vie à définir.

Registre des méthodes et scripts	Décrire les opérations disponibles, leurs préconditions, paramètres, restrictions, versions fonctionnelles et résultats attendus.	Source prévue ; modèle de registre à définir.
Couche d'outils géospatiaux	Exposer des opérations déclarées, valider leurs paramètres, appliquer les politiques d'exécution et normaliser les résultats.	Composant prévu ; catalogue d'outils à établir.
PyQGIS	Réaliser les opérations autorisées qui nécessitent l'accès aux projets, couches, entités ou traitements de QGIS.	Composant prévu ; non présenté comme déployé.
GDAL	Réaliser les opérations autorisées de lecture, écriture, conversion, transformation ou traitement de données géospatiales compatibles.	Composant prévu ; non présenté comme déployé.
PostGIS	Exécuter les opérations spatiales autorisées sur les données accessibles en base, avec contrôle des schémas, requêtes, volumes et droits.	Composant prévu ; non présenté comme déployé.
Retour vers QGIS	Restituer les résultats, couches, messages, actions proposées ou exécutées, sources et alertes nécessitant une décision.	Confirmé par le schéma ; contrat de restitution à préciser.

Responsabilités transversales

L'orchestrateur porte la responsabilité de la cohérence du flux, tandis que chaque outil reste responsable de la validité de ses paramètres et de la sécurité de son exécution. Le RAG fournit du contexte, mais ne remplace ni les contrôles de sécurité ni la validation métier.

Les composants doivent échanger des objets structurés et versionnés plutôt que du texte libre lorsque l'échange déclenche une action. Les champs minimaux attendus sont l'identifiant de la demande, le contexte autorisé, l'intention, le plan, les paramètres, le statut, les avertissements et les références de traçabilité.

13.7.4. Cas d'usage liés aux bâtiments, routes et cours d'eau

Les exemples suivants illustrent des demandes possibles sans supposer l'existence de données particulières, de volumes, de coordonnées ou de résultats réels. Les résultats attendus sont des types de restitution à produire après validation et exécution.

Domaine	Exemple de demande	Résultat attendu	Points de contrôle
Bâtiments	« Sélectionner les bâtiments intersectant l'emprise d'étude et résumer les attributs disponibles. »	Sélection QGIS, tableau de synthèse ou couche dérivée, avec indication des attributs utilisés et des entités exclues.	Emprise, géométrie valide, doublons, CRS et droit de lecture.
Bâtiments et routes	« Identifier les bâtiments situés à proximité du réseau routier selon une distance que je confirmerai. »	Plan demandant ou utilisant la distance validée, puis couche ou sélection annotée dans QGIS.	Distance explicitement confirmée, unité, méthode de proximité et gestion des géométries invalides.

Routes	« Contrôler la continuité géométrique du réseau routier dans l'emprise courante. »	Rapport d'anomalies et éventuellement couche de points ou segments signalés.	Définition de la continuité, tolérance, extrémités attendues et distinction entre anomalie et cas normal.
Routes et cours d'eau	« Repérer les croisements entre les routes et les cours d'eau. »	Résultat spatial des intersections, avec résumé des couches et règles appliquées.	Validité géométrique, dimension des objets, CRS commun et traitement des intersections complexes.
Cours d'eau	« Extraire les cours d'eau dans l'emprise de travail et vérifier les attributs obligatoires disponibles. »	Couche filtrée ou rapport de qualité, sans inventer les attributs absents.	Champs requis définis par une source autorisée, valeurs nulles et statut de la source.

Forme de la restitution

Pour chaque cas, l'assistant doit distinguer la demande initiale, les hypothèses confirmées, les données réellement utilisées, la méthode appliquée, le résultat produit, les avertissements et les sources documentaires citées.

Si la demande ne précise pas un seuil, une emprise, une unité ou une règle métier, le système doit demander une confirmation ou proposer un plan non exécuté. Il ne doit pas compléter silencieusement ces paramètres à partir d'une supposition.

13.7.5. Sécurité, gouvernance et qualité

Thème	Mesures recommandées
Permissions	Appliquer le moindre privilège par utilisateur, projet, couche, outil, schéma et opération. Séparer les droits de lecture, de création, de modification et de suppression.
Validation humaine	Exiger une confirmation pour les changements de données, les traitements coûteux, les sorties sensibles et les actions difficilement réversibles.
Isolation des processus	Exécuter les traitements dans un contexte isolé, limité en accès réseau, fichiers, mémoire, durée et capacité de calcul.
Gestion des secrets	Ne jamais transmettre de secrets au modèle ni les écrire dans les journaux. Utiliser un mécanisme de gestion de secrets adapté à l'environnement choisi.
Requêtes PostGIS	Interdire les requêtes libres non validées, contrôler les schémas, utiliser des paramètres séparés de la logique et limiter les opérations de modification.
Ressources	Définir des limites de temps, de volume, d'emprise, de taille de sortie, de concurrence et de profondeur de planification.
Traçabilité	Associer chaque action à un identifiant, une identité, une demande, un outil, une validation, un statut et un artefact, en excluant les secrets.

Protection des données géographiques	Classer les données, restreindre leur diffusion, contrôler les exports et appliquer les règles de conservation et de suppression établies par l'organisation.
Qualité des données	Contrôler CRS, géométries, schémas, valeurs obligatoires, doublons, précision connue et compatibilité des sources avant et après traitement.

Gouvernance des connaissances

Les sources du RAG doivent avoir un propriétaire, un statut, une date de révision et un périmètre d'utilisation. Une méthode ou un script retiré du registre ne doit plus être proposé pour une nouvelle exécution, même s'il reste présent dans un index ou dans un historique.

Définir les rôles responsables des données, des méthodes, des scripts, des outils et des règles de sécurité.

Définir la procédure de revue et de retrait d'une source documentaire ou d'un outil.

Définir les niveaux de validation requis selon l'impact de l'action.

Définir les règles de conservation des demandes, contextes, journaux et artefacts géographiques.

13.7.6 Vue d'ensemble du fonctionnement

Le fonctionnement combine deux points d'entrée : une demande formulée par l'utilisateur et le contexte fourni par les données et la configuration courante de QGIS. Le système doit signaler les informations manquantes au lieu de déduire silencieusement des paramètres non confirmés.

Flux étape par étape

1. QGIS expose le contexte disponible, notamment les couches Bâtiments, Routes et Cours d'eau, leur état de sélection, leur emprise, leur système de coordonnées et les paramètres explicitement autorisés.
2. L'utilisateur formule une demande, par exemple une analyse, une sélection, une conversion, un contrôle de qualité ou une production de couche dérivée.
3. L'orchestrateur Assistant IA normalise la demande, identifie l'intention, détecte les ambiguïtés et détermine les ressources nécessaires.

4. Le module RAG recherche les passages pertinents dans la base documentaire, le registre des méthodes et le registre des scripts.

5. Modèle (routage Gemma) reçoit la demande normalisée, le contexte QGIS autorisé et les éléments récupérés par le RAG afin de proposer une interprétation et un plan d'action.

6. L'orchestrateur transforme le plan en appels d'outils structurés. Les paramètres sont contrôlés avant toute exécution.

7. La couche d'outils sélectionne l'interface adaptée : PyQGIS pour les opérations liées à QGIS, GDAL pour les traitements de données géospatiales compatibles et PostGIS pour les opérations exécutées sur des données autorisées en base.

8. Les outils exécutent uniquement des opérations déclarées, paramétrées et validées. Ils produisent un résultat structuré, un statut, des avertissements et des références de journalisation.

9. L'orchestrateur vérifie la réponse, associe les sources documentaires et prépare une restitution compréhensible.

10. QGIS reçoit les résultats, les couches ou actions autorisées, le résumé de l'opération, les erreurs éventuelles et les éléments nécessaires à la revue par l'utilisateur.

Règle de séparation

Modèle (routage Gemma) peut proposer une interprétation ou un plan, mais ne doit pas disposer d'un accès implicite à l'exécution. Seule la couche d'outils, après validation du plan et des paramètres, peut déclencher une opération autorisée.

13.7.8. Flux de données et contrats entre composants

Les contrats ci-dessous sont des contrats fonctionnels proposés. Ils décrivent les informations à échanger sans imposer une technologie, un format de sérialisation ou une infrastructure déterminée.

Échange	**Entrées**	**Sorties**	**Validations et erreurs possibles**
QGIS vers orchestrateur	Demande, projet courant, couches autorisées, sélection, emprise, système de coordonnées et préférences explicites.	Contexte normalisé.	Vérifier la disponibilité des couches, les droits de lecture et la cohérence du système de coordonnées ; signaler un contexte incomplet ou contradictoire.
Orchestrateur vers RAG	Question reformulée, intention, périmètre thématique et contraintes de recherche.	Passages, méthodes, scripts référencés et métadonnées de source.	Gérer l'absence de résultat, les sources obsolètes, les conflits documentaires et le dépassement d'un budget de recherche.
Orchestrateur vers Modèle (routage Gemma)	Demande, contexte QGIS autorisé, résultats RAG, règles de sortie et contraintes d'exécution.	Interprétation, hypothèses explicites, plan structuré et réponse provisoire.	Refuser une sortie non structurée, une action non cataloguée, une hypothèse présentée comme un fait ou une demande ambiguë non résolue.

Orchestrateur vers couche d'outils	Nom logique de l'outil, paramètres validés, ressources autorisées, seuils et identifiant de traçabilité.	Résultat normalisé, statut, journaux, avertissements et artefacts.	Contrôler types, valeurs, emprise, CRS, volumes, droits et dépendances ; bloquer les paramètres non conformes.
Couche d'outils vers QGIS	Artefacts produits, résumé, statut et avertissements.	Couche ajoutée ou mise à jour, résultat consultable, action proposée ou message d'erreur.	Vérifier l'intégrité de l'artefact, son origine, sa compatibilité et l'absence d'écrasement non autorisé.

États possibles d'une demande

- **Reçue** la demande et le contexte ont été enregistrés.
- **À préciser** une information nécessaire manque ou plusieurs interprétations sont possibles.
- **Planifiée** un plan d'action structuré a été produit, sans exécution.
- **En attente de validation** une confirmation humaine ou une autorisation supplémentaire est requise.
- **En cours** un outil autorisé est en exécution.
- **Réussie avec avertissements** le résultat est disponible mais doit être interprété avec prudence.
- **Échouée** aucune restitution exploitable ne doit être présentée comme un résultat valide.

13.7.9 Fonctionnement du RAG

Le RAG sert à fournir à l'assistant un contexte vérifiable provenant de sources maîtrisées. Il ne doit pas être considéré comme une preuve automatique de validité : la qualité de la réponse dépend de la qualité, de l'actualité et du périmètre des sources récupérées.

Chaîne de traitement

1. Ingestion: recueillir la documentation, les procédures, les méthodes et les scripts autorisés, puis enregistrer leurs métadonnées, leur statut, leur propriétaire et leur date de révision.

2. Normalisation extraire le texte et les éléments structurés utiles, préserver les titres, les relations entre sections et les avertissements, puis écarter les contenus non autorisés.

3. Découpage: segmenter les sources en unités suffisamment cohérentes pour être retrouvées avec leur contexte. La taille et le recouvrement des segments doivent être évalués sur des exemples représentatifs.

4. Enrichissement: associer à chaque segment des métadonnées telles que le domaine, la méthode concernée, le type d'outil, le statut, la version fonctionnelle et les restrictions d'usage.

5. Indexation : choisir un ou plusieurs mécanismes de recherche selon les besoins. Les options peuvent inclure une recherche textuelle, une recherche sémantique ou une combinaison des deux ; aucune technologie précise n'est imposée ici.

6. Recherche transformer la demande en requête de recherche, appliquer les filtres d'autorisation et récupérer les éléments potentiellement pertinents.

7. Classement et filtrage : ordonner les résultats, éliminer les sources incompatibles ou obsolètes et limiter le contexte transmis au modèle.

8. Génération : demander à Gemma 12B de répondre à partir du contexte récupéré, en séparant les faits sourcés, les hypothèses, les incertitudes et le plan d'action proposé.

9. Citation : retourner les références des sources utilisées, avec suffisamment de précision pour permettre une vérification humaine.

Options d'implémentation à évaluer

Option	**Intérêt**	**Point à évaluer**
Recherche textuelle	Bonne explicabilité sur les termes, noms de méthodes et identifiants.	Sensibilité aux synonymes et aux formulations différentes.
Recherche sémantique	Meilleure tolérance aux variations de vocabulaire et aux formulations naturelles.	Qualité de la représentation, explicabilité et maîtrise des résultats.

Recherche hybride	Combine correspondance lexicale et proximité sémantique.	Complexité de réglage, de classement et d'évaluation.

Règle de citation

Lorsqu'aucune source suffisante n'est retrouvée, l'assistant doit le déclarer clairement. Il ne doit pas fabriquer une référence ni présenter une connaissance générale du modèle comme une procédure validée par l'organisation.

13.7.10 Exécution contrôlée des outils

Principe général

La couche d'outils est une frontière de sécurité entre le raisonnement génératif et les opérations géospatiales. Les outils sont exposés par un catalogue explicite, avec des paramètres déclarés, des préconditions, des sorties attendues et des règles d'autorisation.

1. Sélection de l'outil : l'orchestrateur associé l'intention à un outil enregistré. Si plusieurs outils sont possibles, il conserve les alternatives et justifie le choix.
2. Génération du plan : le plan décrit les étapes, les données d'entrée, les dépendances, les artefacts attendus et les effets potentiellement irréversibles.
3. Validation des paramètres: vérifier les types, les plages, les noms de couches, les systèmes de coordonnées, les emprises, les volumes, les destinations et les droits.
4. Validation métier et humaine : demander une confirmation lorsque l'action modifie des données, produit un volume important, touche une source sensible ou présente un risque difficilement réversible.
5. Préparation isolée : créer un contexte d'exécution limité, avec des ressources, des chemins, des connexions et des permissions strictement nécessaires.
6. Exécution : invoquer l'outil déclaré via PyQGIS, GDAL ou PostGIS, sans permettre au modèle de modifier directement la commande finale.
7. Journalisation: enregistrer l'outil, les paramètres non secrets, l'identité de l'appelant, les validations, le statut, la durée, les volumes et les références d'artefacts.
8. Contrôle du résultat: vérifier le statut, le schéma, le CRS, les contraintes d'intégrité et les avertissements avant de déclarer le résultat exploitable.

9. Retour envoyer vers QGIS le résultat, les actions réalisées, les limites connues et les étapes restant à la charge de l'utilisateur.

Interdiction impérative

Le système ne doit jamais exécuter directement du code, un script ou une requête généré par Modèle (routage Gemma) ou fourni par l'utilisateur sans passer par un outil enregistré, une validation des paramètres, un contrôle des permissions et un mécanisme d'exécution autorisé.

Exemples de contrôles spécifiques

- **PyQGIS** limiter les classes, traitements et opérations accessibles au catalogue approuvé.
- **GDAL** contrôler les formats, chemins, pilotes, emprises, tailles de sortie et options susceptibles de modifier des données.
- **PostGIS** utiliser des requêtes paramétrées ou des opérations préenregistrées, contrôler les schémas accessibles et empêcher les instructions non autorisées.

13.7.11 Observabilité et gestion des erreurs

Éléments à observer

Zone	Informations à suivre	Utilité
Demande	Identifiant, identité autorisée, contenu ou empreinte selon les règles de confidentialité, contexte QGIS et statut.	Reconstituer le parcours d'une demande et détecter les abandons.
RAG	Requête, filtres, sources récupérées, scores ou rangs disponibles, sources écartées et motif d'exclusion.	Évaluer la pertinence et expliquer une réponse.

Modèle	Version fonctionnelle déclarée, durée, statut, longueur des entrées et sorties, refus et erreurs de format.	Détecter les variations de comportement sans journaliser de données interdites.
Outils	Nom, paramètres contrôlés, validations, durée, ressources, statut, avertissements et artefacts.	Auditer l'exécution et diagnostiquer les incidents.
QGIS	Action de restitution, couche ciblée, résultat affiché, erreur d'intégration et confirmation utilisateur.	Vérifier que le résultat présenté correspond au résultat produit.

Catégories d'erreurs

- **Entrée invalide** demande incomplète, couche absente, type incompatible, emprise vide ou CRS incohérent.
- **Contexte insuffisant** aucune source RAG pertinente, méthode non documentée ou information obligatoire manquante.
- **Autorisation refusée** utilisateur, couche, schéma, outil ou opération non autorisé.
- **Plan invalide** outil inexistant, dépendance incohérente, paramètre non validé ou effet non acceptable.
- **Échec d'exécution** erreur PyQGIS, GDAL ou PostGIS, dépassement de limite, géométrie invalide ou ressource indisponible.
- **Résultat douteux** sortie produite mais contrôles de qualité incomplets, avertissements non résolus ou intégrité non démontrée.

Comportement attendu

Une erreur doit être associée à un statut explicite, un message compréhensible, un identifiant de corrélation et, lorsque cela est possible, une action corrective. Le système doit éviter les reprises automatiques non bornées et ne doit pas relancer une opération susceptible de produire des doublons ou des effets répétés sans confirmation.

13.7.12 Déploiement et configuration

La conception reste agnostique sur l'infrastructure. Les environnements doivent être séparés par configuration, données, secrets, permissions, journaux et règles d'exécution, sans présumer d'un mode d'hébergement particulier.

Environnement	**Objectif**	**Configuration et garde-fous**
Développement	Construire les composants, expérimenter les contrats et utiliser des jeux de données de test contrôlés.	Permissions limitées, données non sensibles ou anonymisées, outils simulés si nécessaire, journaux détaillés et validation fréquente.
Test	Vérifier l'intégration QGIS/outils, le RAG, la sécurité et les scénarios représentatifs.	Configuration proche de la cible sans reprendre les secrets ni les données de production ; exécutions reproductibles et jeux de référence versionnés.
Production	Fournir le service aux utilisateurs autorisés avec des contrôles opérationnels.	Permissions minimales, secrets séparés, approbations, limites de ressources, sauvegarde ou conservation définie, supervision et procédure de retour arrière.

Paramètres à gérer explicitement

- **Modèle** référence fonctionnelle de Gemma 12B ou 4B, règles de génération, format de sortie et seuils de rejet des réponses non structurées.
- **RAG** sources actives, filtres de droits, politique de fraîcheur, taille de contexte, règles de citation et stratégie de repli.
- **Outils** catalogue actif, paramètres autorisés, préconditions, limites de ressources, niveaux de confirmation et règles d'écriture.

- **QGIS** couches exposées, actions de restitution, gestion des sélections, format des artefacts et comportement en cas d'erreur.
- **PostGIS** connexions autorisées, schémas accessibles, opérations disponibles, limites de volume et contrôle des modifications.
- **Observabilité** niveaux de journalisation, conservation, alertes, corrélation et accès aux traces.

Gestion de configuration

Les valeurs spécifiques à un environnement doivent être séparées des règles fonctionnelles et du code des outils. Toute modification du catalogue, des permissions, des sources RAG ou des limites d'exécution doit être revue, versionnée et traçable.

13.7.13 TESTS

Type de test	Objectif	Exemples de vérifications
Tests unitaires	Vérifier les fonctions isolées de normalisation, validation, planification, filtrage et formatage.	Paramètres manquants, CRS incohérent, outil inconnu, statut d'erreur et génération de contrats conformes.
Tests d'intégration QGIS/outils	Vérifier les échanges entre QGIS, l'orchestrateur et la couche d'outils.	Lecture des couches, sélection, création d'artefact, retour d'erreur et absence d'écriture non autorisée.
Tests du RAG	Vérifier la récupération, la fraîcheur, les filtres d'accès et les citations.	Questions de référence, sources concurrentes, absence de résultat, document retiré et méthode hors périmètre.

Tests de sécurité	Vérifier la résistance aux instructions malveillantes, aux dépassements de droits et aux entrées dangereuses.	Tentative d'exécution directe, injection dans une requête, accès à un schéma interdit, extraction d'un secret et contournement d'une validation.
Tests de non-régression	Garantir la stabilité des réponses structurées, des contrats et des opérations existantes.	Scénarios de référence sur bâtiments, routes et cours d'eau, sorties attendues, erreurs connues et citations.

Jeux de tests

"Éléments chronophages mais étape la plus importante du projet".

Les jeux de tests doivent contenir des cas nominaux, des données incomplètes, des géométries invalides, des couches vides, des CRS différents, des demandes ambiguës, des autorisations insuffisantes et des opérations produisant des avertissements. Ils doivent éviter toute donnée géographique sensible non nécessaire.

Versionner les demandes, contextes, plans et résultats attendus des scénarios de référence.

Tester séparément le comportement du modèle, la pertinence du RAG et la sécurité de la couche d'outils.

Vérifier qu'un échec ne produit pas d'artefact présenté comme valide dans QGIS.

Tester la reprise après interruption et l'absence de répétition dangereuse d'une opération.

13.7.14 Limites, risques et décisions techniques à prendre

Limite ou risque	**Conséquence possible**	**Décision à prendre**
Ambiguïté du langage naturel	Plan incorrect ou action fondée sur une hypothèse implicite.	Définir les informations obligatoires et les seuils déclenchant une demande de clarification.

Réponse générative inexacte	Explication convaincante mais non conforme à la documentation ou aux données.	Définir les formats structurés, validations, citations et règles de refus.
RAG incomplet ou obsolète	Méthode incorrecte, outil retiré ou règle métier manquante.	Définir la gouvernance des sources, la fraîcheur et les tests de récupération.
Qualité variable des géométries	Résultat incomplet, erreur d'outil ou interprétation spatiale incorrecte.	Définir les contrôles et les politiques de correction ou d'exclusion.
Coût et durée des traitements	Blocage, dépassement de ressources ou expérience utilisateur dégradée.	Définir les limites, l'annulation, la reprise et les traitements asynchrones éventuels.
Accès aux données sensibles	Divulgation, export ou traitement non autorisé de données géographiques.	Définir la classification, les droits, la conservation et les contrôles de sortie.
Évolutivité du catalogue d'outils	Surface d'attaque et complexité de validation croissantes.	Définir un processus d'homologation, de retrait et de compatibilité des outils.

Décisions structurantes

- **Réversibilité** distinguer les opérations de lecture ou d'analyse des opérations qui modifient ou suppriment des données.
- **Autorité de la source** établir quelles sources priment lorsqu'une documentation, une méthode et une configuration se contredisent.
- **Validation humaine** déterminer les actions qui peuvent être exécutées automatiquement et celles qui exigent une confirmation.

- **Contrat de restitution** décider si QGIS reçoit une couche, une sélection, un rapport, une action proposée ou plusieurs de ces éléments.
- **Stratégie d'exécution** choisir le niveau d'isolation et le mode de gestion des traitements longs sans présumer d'une infrastructure.

13.7.15 Feuille de route recommandée

1. **Phase 1 Cadrage et contrats** : confirmer les rôles, les cas d'usage prioritaires, les couches exposées, les niveaux de risque, les contrats d'échange et les règles de validation.
2. **Phase 2 Catalogue minimal d'outils** : définir un petit ensemble d'opérations en lecture ou à faible risque, avec paramètres, préconditions, résultats et erreurs normalisées.
3. **Phase 3 Orchestration contrôlée** : mettre en place le cycle demande, interprétation, plan, validation, exécution, journalisation et restitution, avec des outils simulés si nécessaire.
4. **Phase 4 RAG maîtrisé** : organiser la documentation, les méthodes et les scripts, tester le découpage, comparer les options de recherche et imposer les citations.
5. **Phase 5 Intégration QGIS** : connecter le contexte des couches Bâtiments, Routes et Cours d'eau, puis restituer les sélections, couches dérivées, rapports et demandes de confirmation.
6. **Phase 6 Sécurité et gouvernance** : appliquer les permissions, l'isolation, la gestion des secrets, les limites de ressources, le contrôle PostGIS et la traçabilité.
7. **Phase 7 Validation élargie** : exécuter les tests d'intégration, RAG, sécurité et non-régression sur des jeux contrôlés et représentatifs.
8. **Phase 8 Mise en service progressive** : ouvrir l'accès par groupes ou scénarios, observer les erreurs, recueillir les retours et élargir le catalogue après revue.

Critères de passage entre phases

Critère	Attendu
Contrats	Les entrées, sorties, validations et erreurs sont documentées et testables.

Outils	Chaque outil actif possède un propriétaire, un périmètre, des paramètres validés et une procédure de retrait.
Sécurité	Les accès, secrets, limites, journaux et validations sont vérifiés sur des scénarios négatifs.
RAG	Les sources autorisées sont identifiables, citées et testées sur des questions représentatives.
QGIS	Les résultats et erreurs sont restitués sans ambiguïté et sans écrasement non autorisé.

“voir Annexe 1 “Architecture Multi AI”, mode de fonctionnement sur un test équivalent

et Annexe 2 scénario d’exécution et pseudo-flux d’orchestration”

13.7.16 Vers une IA Autonome et Durable

D'ici la fin de l'année 2026, l'IA décentralisée est devenue la norme architecturale. La convergence entre la démocratisation du matériel (Apple Silicon, NVIDIA RTX), la maturité des outils comme Ollama et l'impératif souverain a créé un écosystème où chaque terminal est un nœud de décision autonome.

La transformation est profonde : l'IA n'est plus un service distant loué à prix d'or, mais un utilitaire local, sécurisé et durable. Pour l'architecte comme pour l'entreprise, cette autonomie est la clé de l'innovation dans un monde où la donnée est devenue le bien le plus précieux et le plus vulnérable.

14. L’automatisation ultime et la création d’une équipe IA

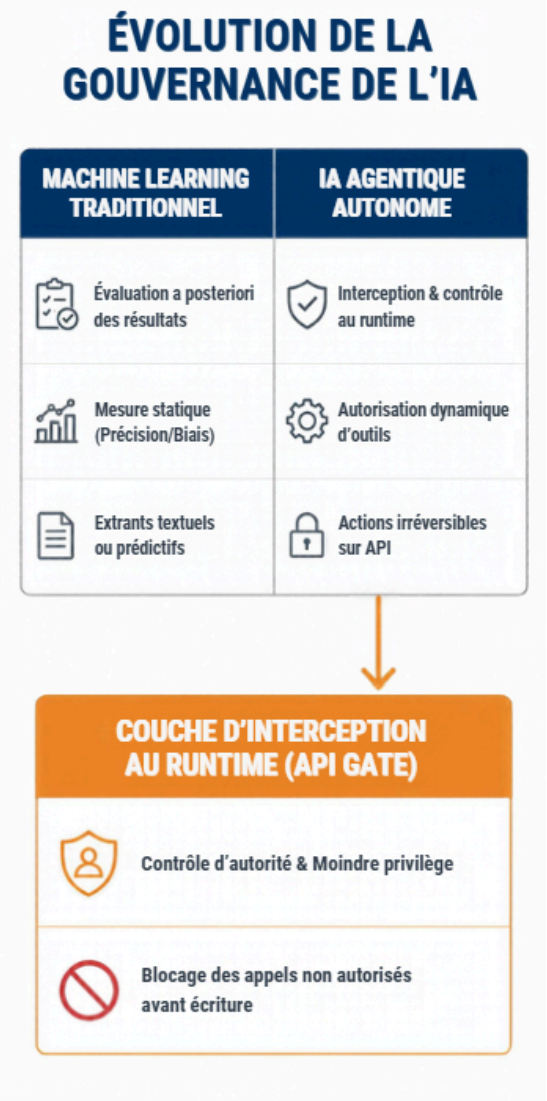
Avant tout, nous entrons dans une architecture nouvelle qui nécessite une attention particulière et peut devenir une faille de sécurité si elle est mal mise en place. Nos tests ont été effectués en local pour la plupart et sur une machine dédiée. Les tests ont été fait avec Hermes agent. Le choix s’est porté sur la sécurisation des skills et la capacité à apprendre automatiquement.

14.1 La Recette d'Établissement d'une Équipe de Géomaticiens Autonomes Propulsée par l'IA Agentique

14.1.1 Thématique & Cadre Stratégique de l'Autonomie

La géomatique moderne franchit actuellement un cap d'ingénierie critique. Historiquement restreints à la numérisation manuelle, à la correction vectorielle et à l'exécution séquentielle de scripts de géotraitement au sein de Systèmes d'Information Géographique (SIG) traditionnels, les services géospatiaux font face à une accélération massive du volume de données spatio-temporelles non structurées (flux d'imagerie, flux IoT, capteurs Lidar, rapports textuels géolocalisés). Dans cet écosystème à haute fréquence, la transition de l'automatisation rigide basée sur des scripts Python déterministes et fragiles aux variations de schémas vers une autonomie décisionnelle pilotée par des agents logiciels ne relève pas d'une simple recherche d'efficience opérationnelle : elle constitue un saut paradigmatique fondamental. L'agent logiciel autonome ne se contente pas d'exécuter une règle préprogrammée ; il appréhende une consigne métier à haut niveau d'abstraction, planifie une séquence d'outils géospatiaux, réagit aux exceptions attributaires ou topologiques et maintient un état conversationnel persistant 24 heures sur 24 et 7 jours sur 7.

Cette mutation exige une refonte radicale du cadre de gouvernance des systèmes intelligents en entreprise. La gouvernance du Machine Learning (ML) traditionnel s'est structurée autour de l'évaluation a posteriori d'extrants statiques, mesurant la précision globale d'un modèle de classification, l'équité d'un score ou l'absence de biais dans un échantillon d'entraînement. À l'inverse, la gouvernance de l'IA agentique déplace le point de contrôle de l'extrant textuel vers l'exécution d'actions autonomes en temps réel sur des API, des bases de données spatiales de production (PostGIS, Enterprise Geodatabase) et des systèmes d'enregistrement. Il ne s'agit plus de vérifier a posteriori si la prédiction d'un modèle statistique est plausible, mais de valider, au millième de seconde précédant l'appel système, si un agent dispose de l'autorité légitime pour purger une table cadastre, altérer la topologie d'un réseau ou soumettre un rapport d'aménagement du territoire.



Les impacts opérationnels des défaillances agentiques en milieu professionnel dépassent largement la simple « hallucination textuelle ». Une étude menée par Forrester en 2025 démontre ainsi que **72 % des échecs d'IA en entreprise découlent d'actions système non autorisées** et d'appels d'outils hors périmètre plutôt que de fautes de syntaxe ou d'erreurs de génération de texte. Dans le secteur géospatial, où la donnée possède une valeur juridique, patrimoniale et environnementale majeure, l'exécution non contrôlée d'un agent peut conduire à la suppression accidentelle de couches d'infrastructures critiques, à la fuite de données foncières nominatives ou à la dégradation silencieuse d'index spatiaux en production.

La thèse centrale de cet ouvrage établit que la « recette » industrielle d'une équipe de géomaticiens autonomes ne repose pas sur la seule adoption de modèles de langage (Large Language Models ou LLM), mais sur l'articulation indissociable de trois piliers d'architecture :

1. **Un moteur d'agent persistant** (Runtime) garantissant la haute disponibilité, l'asynchronisme et la persistance contextuelle.

2. **Un outillage métier géospatial avancé** couplant apprentissage profond (Deep Learning), statistiques spatiales et analyse explicite de la géographie.

3. Un cadre de **gouvernance** structurel et nativement intégré capable de borner l'autonomie à l'exécution et de bloquer les dérives fonctionnelles au point d'impact système.

La mise en œuvre de cette vision stratégique nécessite, en premier lieu, un ancrage infrastructurel irréprochable au niveau du moteur d'exécution et du serveur d'agent.

14.1.2 Le Socle Infrastructurel : Architecture Moteur & Runtime de l'Agent

L'institutionnalisation d'une équipe géomatique autonome impose un choix strict quant au mode d'hébergement et d'orchestration du moteur d'exécution (runtime). Garantir une disponibilité 24/7 de la force de travail agentique tout en protégeant les données géospatiales internes interdit l'usage d'architectures empiriques ou non isolées.

L'exécution d'agents autonomes sur des postes de travail personnels ou des ordinateurs portables introduit des failles de sécurité et de continuité d'activité inacceptables. Sur un poste client, le cycle de vie de l'agent est subordonné aux fermetures de session, à la mise en veille matérielle et aux fluctuations du réseau local, brisant irrémédiablement l'exécution des tâches planifiées d'arrière-plan (Cron jobs). Sur le plan de la sécurité applicative, un agent s'exécutant en local dispose d'une visibilité directe sur le système de fichiers de l'utilisateur, ses jetons de session et ses dossiers personnels. Lorsqu'un tel agent parcourt le Web à la recherche de données cartographiques ou ingère des documents PDF non nettoyés provenant de sources externes, il devient vulnérable aux attaques par injection d'invites (prompt injection indirecte). Une instruction malveillante dissimulée dans les métadonnées d'un fichier GeoTIFF ou dans le texte d'un site web téléchargé peut détourner le raisonnement du LLM et l'inciter à lire, exfiltrer ou supprimer des données sensibles stockées sur la machine hôte.

À l'inverse, l'hébergement du moteur d'agent sur un Serveur Virtuel Privé (VPS) ou une infrastructure cloud conteneurisée sous Docker confine l'agent dans un bac à sable (sandbox) strictement isolé. En cas d'injection malveillante, le rayon d'impact est circonscrit au conteneur, garantissant la protection intégrale du réseau interne de l'entreprise et la persistance H24 des opérations.

Le socle logiciel sélectionné pour piloter cette autonomie repose sur l'architecture open-source **Hermes Agent** (développée par Nous Research). Hermes Agent s'exécute en tant que démon système d'arrière-plan et expose une interface API HTTP compatible avec les standards de l'industrie (par défaut sur le port 8642, activé via API_SERVER_ENABLED=true et sécurisé par authentification Bearer via API_SERVER_KEY). L'architecture API d'Hermes s'articule autour de trois points d'accès majeurs aux caractéristiques sémantiques distinctes :

- POST /v1/chat/completions : Point d'accès sans état (*stateless*) conforme au format OpenAI Chat Completions. Chaque requête véhicule l'intégralité du tableau de messages. Pour prendre en charge les traitements géospatiaux très longs (ex: génération d'un modèle numérique de terrain) sans

provoquer de rupture par délai d'attente (*idle timeout*) sur les clients ou proxys, le flux SSE (*Server-Sent Events*) émet une ligne de commentaire : keepalive toutes les 10 secondes.

- POST /v1/responses : Point d'accès avec gestion d'état (*stateful*) basé sur l'API Responses. Le serveur prend en charge la persistance de l'historique complet de la conversation (incluant les appels d'outils et leurs résultats texte/JSON) via le paramètre previous_response_id ou via le paramètre conversation (nommage explicite d'une chaîne de dialogue). Le client se trouve ainsi dispensé de renvoyer l'historique volumineux des exécutions d'outils à chaque tour.
- POST /v1/runs : Point d'accès asynchrone dédié à la soumission de requêtes à longue durée d'exécution. Le serveur enregistre l'admission de la tâche et retourne immédiatement un run_id. Le client ou l'interface d'administration peut interroger l'état de la tâche (GET /v1/runs/{run_id}), s'abonner au flux d'événements en direct (GET /v1/runs/{run_id}/events), interrompre l'exécution (POST /v1/runs/{run_id}/stop) ou soumettre une validation explicite (POST /v1/runs/{run_id}/approval).

Afin d'assurer la sécurité des réessais réseau sur l'API des requêtes asynchrones, l'endpoint POST /v1/runs prend en charge l'en-tête Idempotency-Key (composé de 1 à 255 caractères ASCII visibles). Lorsqu'une clé d'idempotence est soumise, le serveur réserve immuablement le run_id correspondant dans son registre pendant 24 heures. En cas de réessai exact comportant une clé et un corps de requête identiques, le serveur retourne le run_id initial avec le statut HTTP 202 Accepted et l'en-tête HTTP Idempotency-Replayed: true, sans relancer le traitement. Si la même clé est réutilisée avec un corps JSON différent, l'API rejette la demande avec le statut HTTP 409 Conflict et le code d'erreur idempotency_key_conflict.

La mémoire persistante à long terme d'Hermes Agent repose sur un cycle d'apprentissage fermé (*closed learning loop*). Le système couple une base de recherche textuelle SQLite FTS5 (*Full-Text Search*) à un moteur de modélisation dialectique de l'utilisateur (Honcho). Afin de maintenir le cloisonnement strict des mémoires lors de l'utilisation de canaux de communication multi-utilisateurs ou partagés (tels que Telegram, Open WebUI ou Microsoft Teams), le serveur s'appuie sur l'en-tête HTTP spécifique X-Hermes-Session-Key.

Tandis que l'en-tête X-Hermes-Session-Id définit la portée d'une transcription courante (et bascule lors de la création d'un nouveau dialogue), la clé X-Hermes-Session-Key (d'une longueur maximale de 256 caractères et rejetant rigoureusement les caractères de contrôle \r, \n, \x00) définit l'identifiant stable de l'utilisateur ou du canal (ex: agent:gis:webui:user-88). Cette valeur est transmise au fournisseur de mémoire Honcho pour garantir que l'apprentissage sur les préférences utilisateur persiste à travers les réinitialisations de conversations. Le serveur réémet systématiquement l'en-tête X-Hermes-Session-Key dans ses réponses JSON et flux SSE afin d'attester la portée de mémoire appliquée.

Critère d'évaluation,Application Desktop / Ordinateur Portable,Serveur / VPS Dédié (Conteneurisé)

Disponibilité opérationnelle,Interrompue dès la mise en veille matérielle ou la déconnexion du poste client."Continue (24/7/365), adaptée à la planification Cron et aux surveillances d'arrière-plan."

Ressources / Coût,Exploite le matériel local ; aucun surcoût d'hébergement direct mais instabilité d'exécution.Ressources système garanties (ex. VPS KVM) ; coût d'infrastructure résiduel (~5 à 20 $/mois).

Sécurité face aux injections,"Risque critique : accès direct au système de fichiers local, aux identifiants et données privées.",Élevée : isolation étanche dans un conteneur Docker ; rayon d'impact strictement confiné.

Maintien de la mémoire,Fragmenté et dépendant du maintien de la session locale et de l'état de l'application client.

Une fois ce socle d'exécution serveur stabilisé, il devient possible de raccorder le moteur d'agent aux bibliothèques métiers d'analyse géospatiale et aux frameworks assistant IA.

14.1.3 L'Outillage Métier Géospatial & l'Intégration de l'agent

L'IA agentique ne délivre sa pleine valeur métier qu'à la condition de s'interfacer nativement avec des moteurs de géotraitement de niveau industriel (tels qu'ArcGIS Pro et son environnement Python ArcPy ou QGIS avec PyGIS). Cette intégration convertit des consignes abstraites en chaînes analytiques automatisées applicables sur des rasters, des vecteurs et des structures multidimensionnelles.

Sur le plan opérationnel, la connexion entre le moteur d'agent et l'écosystème SIG est pilotée par la passerelle d'outils (*Tool Gateway*) d'Hermes Agent. Lorsque l'agent décide d'exécuter un traitement géospatiale, il génère un script Python ArcPy ou fait appel à une fonction préconfigurée qu'il exécute au sein du sous-système terminal de son conteneur Docker isolé. Pour les chaînes de géotraitement lourdes (comme une régression géographiquement pondérée ou une classification de Nuage de points Lidar), la passerelle déporte le calcul via l'API asynchrone /v1/runs. L'agent surveille la progression du processus, capture le flux d'extrant standard (*stdout*), les journaux d'erreurs (*stderr*) ainsi que les objets JSON de résultats géométriques ou attributaires, avant d'intégrer ces données dans sa mémoire procédurale.

L'intégration de l'assistant IA et du Deep Learning offre à l'agent la capacité de structurer de l'information géospatiale à partir de sources hétérogènes. Dans le domaine de la télédétection, l'agent orchestre des modèles d'apprentissage profond pour réaliser la classification de pixels, la segmentation d'images et la détection d'objets sur des rasters haute résolution (par exemple, le détourage automatisé du bâti ou le suivi du couvert végétal). Sur les données tridimensionnelles, il exploite les boîtes d'outils 3D pour classifier les nuages de points Lidar et dériver des modèles numériques d'élévation. Pour les flux non structurés textuels (rapports d'aménagement, arrêtés préfectoraux, flux d'actualités), l'agent déploie des techniques de traitement du langage naturel (NLP) pour extraire les entités nommées locales et les adresses, alimentant ainsi des moteurs de géocodage.

En outre, l'agent s'appuie sur des algorithmes statistiques **explicitement spatiaux** . Contrairement aux méthodes de Machine Learning génériques qui traitent les observations comme des entités indépendantes, ces modèles intègrent directement dans leurs équations mathématiques les notions de proximité, de forme, de topologie et de distribution spatiale. L'agent déploie ainsi des outils d'analyse des modèles spatio-temporels (*space-time pattern mining*), des prévisions de séries chronologiques et des régressions pondérées pour identifier des agrégats (*clusters*), détecter des anomalies territoriales et établir des projections cartographiques.

Un pilier fondamental de cette architecture réside dans la **mémoire procédurale** et le système de création autonome de compétences (*skills system*). Lorsqu'un agent résout avec succès un problème d'ingénierie

spatiale complexe par exemple, une séquence articulant le découpage d'un raster, l'extraction de pentes et l'agrégation statistique par bassin versant , il consigne cette chaîne d'opérations sous la forme d'une procédure structurée et réutilisable. Strictement conforme à la norme open-source agentskills.io, cette nouvelle compétence est documentée avec ses paramètres d'entrée/sortie et sauvegardée. Elle est ensuite publiée sur le *Skills Hub* de l'organisation, devenant immédiatement utilisable par l'ensemble des autres agents de l'équipe.

L'équipe géomatique autonome s'appuie sur un outillage analytique structuré :

- **Boîte à outils Statistiques Spatiales (*Spatial Statistics*)** : Jeux d'outils *Appariement d'agrégats* (clustering spatial) et *Modélisation de relations spatiales* (régression géographiquement pondérée prenant en compte la forme et la proximité).
- **Boîte à outils IA GIS** : Jeux d'outils *Analyse de texte* (extraction NLP pour le géocodage), *Entité et analyse tabulaire* , et *IA de série chronologique* .
- **Boîte à outils Spatial Analyst** : Outils d'algèbre spatial, d'analyse multidimensionnelle sur rasters spatio-temporels et modélisateur d'aptitude (*Suitability Modeler*).
- **Boîte à outils Image Analyst** : Jeux d'outils *Segmentation et classification* et *Deep Learning* (détection d'objets, classification de pixels et simulation).
- **Boîte à outils 3D Analyst** : Jeu d'outils *Nuage de points* (classification, filtrage et interpolation de données Lidar 3D).

L'attribution d'outils d'une telle puissance d'exécution exige, en contrepartie, la mise en place d'un cadre de gouvernance et de contrôle d'autonomie d'une rigueur absolue.

14.1.4 Cadre de Gouvernance Agentique, Sécurité & Contrôle d'Autonomie

Déployer des agents autonomes connectés à des SIG d'entreprise exige l'instauration d'une « autonomie gouvernée ». Ce concept impose que toute action générée par un modèle de langage soit interceptée, évaluée et validée par un cadre de contrôle déterministe au moment même de son exécution (*runtime*), bloquant l'instruction au niveau de la couche d'orchestration avant qu'elle ne modifie l'environnement de production.

Le référentiel de gouvernance de l'équipe géomatique autonome repose sur **4 piliers fondamentaux** :

1. **Identité et gestion des accès à moindre privilège** : Chaque agent reçoit une identité machine unique et vérifiable. Le partage de clés d'API globales ou l'usurpation de comptes utilisateurs humains est strictement proscrit. Les autorisations d'écriture sur les bases géographiques sont restreintes au périmètre exact de la mission assignée à l'agent.
2. **Contrôles d'exécution et garde-fous déterministes** : L'application des règles de sécurité est exécutée par le moteur d'orchestration de manière totalement indépendante de la couche de raisonnement du LLM. Les garde-fous insérés dans le système de prompt du modèle étant vulnérables aux injections d'invites, la passerelle d'exécution (*Runtime Gate*) valide

déterministement les commandes (scripts Python, requêtes SQL) avant leur transmission au système hôte.

3. **Autonomie gouvernée et intervention humaine (*Human-in-the-loop*)** : L'autonomie s'exerce selon un modèle de maturité gradué en fonction du risque. Les opérations en lecture seule ou les traitements cartographiques temporaires s'exécutent de manière totalement autonome. À l'inverse, les actions irréversibles ou à fort impact (suppression de tables, mise à jour du cadastre) sont automatiquement interceptées et mises en attente dans des files d'approbation humaines via l'endpoint /v1/runs/{run_id}/approval.

4. **Traçabilité et journalisation immuable** : L'infrastructure génère un registre d'audit continu et non modifiable consignant l'ensemble des appels d'outils, les arguments transmis, la clé de session et le résultat renvoyé. L'implémentation de cette observabilité native permet de **réduire de 50 % les délais d'audit de conformité** dans les organisations déployant des agents gouvernés.

Ce référentiel s'articule directement avec les cadres normatifs et réglementaires internationaux :

- ***Règlement européen sur l'IA (AI Act)** : Conformité avec l'Article 14 imposant une supervision humaine effective sur les systèmes autonomes, et avec l'Article 50 exigeant la transparence des opérations automatisées.*
- ***NIST AI RMF (Risk Management Framework)** : Alignement sur le profil d'IA générative concernant la maîtrise des risques d'accès non contrôlé aux outils systèmes et d'escalade d'autorité.*
- ***Top 10 OWASP pour l'IA Agentique** : Déploiement de contre-mesures ciblant la surautorisation (excessive agency), le détournement d'outils et les injections d'invites indirectes.*

Risque d'autonomie, Cause technique sous-jacente, Contrôle structurel de prévention, Surautorisation (Prolifération d'autorité), Octroi de privilèges système permanents et étendus par facilité de configuration, Identités machines isolées appliquées selon le principe du moindre privilège strict.

Défaillances en cascade, Propagation d'une donnée spatiale corrompue entre agents au sein d'une chaîne de traitement ,"Isolation des sous-agents, validation des interfaces et vérification des données transmises."

Injection d'invites & Contournement,"Instructions malveillantes dissimulées dans des rasters, vectoriels ou fichiers Web ingérés.","Validation et filtrage des commandes dans la couche d'orchestration, de façon externe au LLM."

Lacunes de traçabilité, Fragmentation des journaux lors d'exécutions multi-agents distribuées ,Journalisation centralisée immuable liée à la clé de session stable (X-Hermes-Session-Key).

Dilemme de la responsabilité, Incertitude quant au responsable humain lors d'une altération de données par un agent, Attribution formelle de superviseurs humains et routage automatique vers la file d'approbation.

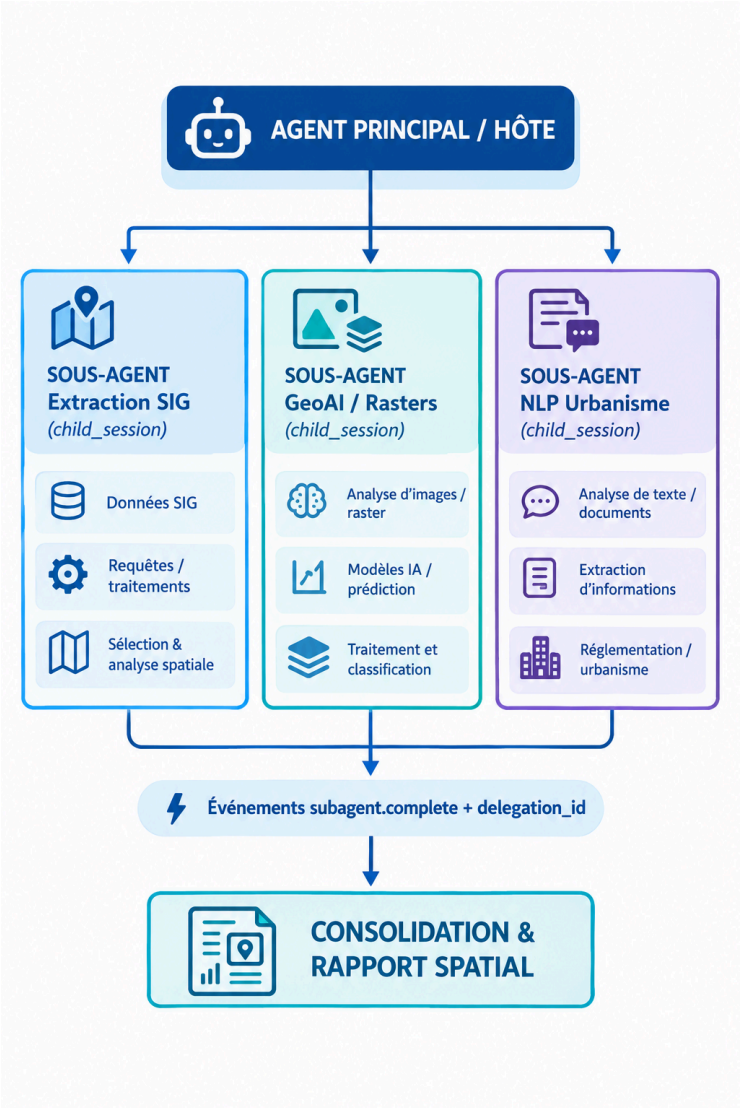
La sécurisation du cadre d'exécution offre le niveau de confiance nécessaire pour orchestrer le travail parallèle de plusieurs agents et optimiser les coûts de traitement.

14.1.5 Orchestration Multi-Agents, Automatisation Planifiée & Optimisation Économique

Maximiser la productivité d'une équipe géomatique autonome repose sur la capacité de l'architecture à paralléliser les traitements complexes au moyen de sous-agents spécialisés, tout en maîtrisant les coûts d'inférence LLM.

Lorsqu'une instruction complexe est confiée à l'agent principal (par exemple : « Réaliser un diagnostic d'aménagement sur un nouveau corridor de transport »), le moteur utilise un modèle de **délégation dynamique** . L'agent principal scinde la consigne en sous-tâches indépendantes et instancie simultanément plusieurs sous-agents spécialisés (pattern *fan-out*). Un premier sous-agent exécute l'extraction des réseaux routiers via PyGIS, un second effectue la segmentation d'imagerie satellitaire par Deep Learning, tandis qu'un troisième analyse les documents d'urbanisme en NLP.

Pendant cette phase, le flux SSE de l'agent principal émet les événements de structure subagent.start et subagent.complete. Ces événements transmettent le child_session_id ainsi que le delegation_id du lot d'exécution, permettant à l'agent hôte et aux interfaces d'orchestration de suivre la durée, le coût en jetons et l'état de chaque sous-agent, sans polluer le flux de conversation principal avec du bruit d'exécution intermédiaire.



L'automatisation continue s'appuie sur le gestionnaire de tâches planifiées (*Cron / Scheduled Jobs*), accessible via l'API REST sous les routes /api/jobs. Un job configure un prompt, une fréquence d'exécution, un jeu de compétences métiers et un canal de livraison cible (Telegram, Teams, Slack). Configuré sur son canal principal via la commande /set home, l'agent exécute de manière autonome des vannes de contrôle automatisées : surveillance continue de flux WFS/WMS, mise à jour nocturne d'index spatiaux et envoi proactif d'alertes cartographiques sur la messagerie de l'équipe SIG.

Sur le plan économique, le fonctionnement d'un agent autonome génère un volume massif de jetons. À chaque tour de rôle, le moteur réinjecte dans le modèle l'ensemble du prompt système, la description de dizaines d'outils géospatiaux, l'historique de conversation et les contextes de mémoire. Une requête d'agent déploient des outils complexes peut ainsi consommer plus de 38 000 jetons d'entrée (*prompt tokens*) par tour.

Pour garantir la viabilité économique du système, l'architecture associe l'usage de modèles d'inférence performants et très économiques (tels que DeepSeek-V4-Flash via la passerelle OpenRouter, ou un abonnement Nous Portal) à la technique de **mise en cache du contexte** (*prompt caching*). Le fournisseur détecte la portion immuable du prompt (instructions système, définitions d'outils) et la conserve en

mémoire cache. L'API d'Hermes Agent restitue cette optimisation dans ses métriques en isolant les jetons lus en cache (usage.cache_read_tokens) des jetons écrits (usage.cache_write_tokens). Les jetons lus en cache bénéficiant d'une réduction tarifaire atteignant 75 %, le coût moyen par exécution s'en trouve drastiquement réduit.

Toutefois, le risque majeur d'explosion des coûts et de dégradation des performances en géomatique découle de la taille des données spatiales renvoyées par les outils. Une seule requête SQL/Spatial renvoyant des entités GeoJSON complexes peut générer plusieurs mégaoctets de texte. Si ce résultat est conservé verbatim dans la base SQLite du serveur (response_store.db), le chaînage des réponses via previous_response_id réinjecte ce volume massif de données à chaque tour, provoquant une saturation mémoire et une explosion de la consommation de jetons.

Pour parer à ce risque, l'architecture doit impérativement configurer le paramètre de plafonnement de l'historique des outils dans le fichier de configuration du serveur d'agent :

```
gateway:
  api_server:
    enabled: true
    port: 8642
    history_tool_output_max_chars: 2000 # Plafond en caractères des
sorties d'outils stockées
```

En fixant history_tool_output_max_chars à une valeur strictement supérieure à 0 (par exemple 2 000 caractères), le serveur d'agent tronque automatiquement les sorties d'outils volumineuses stockées dans la base de données d'historique (en conservant le début de la réponse suivi de la mention ...N more chars). Ce tronquage s'applique exclusivement à la mémoire d'historique stockée pour les tours suivants, sans altérer le résultat immédiat renvoyé à l'utilisateur ni l'exécution réelle du script. Ce garde-fous garantit que la base de données response_store.db et la fenêtre de contexte restent d'une taille optimale lors des chaînages conversationnels longs.

La maîtrise de l'orchestration multi-agents et des coûts d'inférence reconfigure en profondeur les missions et la posture de l'ingénieur géomaticien.

14.1.6. Feuille de Route Opérationnelle & Redéfinition du Rôle du Géomaticien

L'avènement de l'IA agentique ne marque pas la disparition du géomaticien humain, mais opère une élévation majeure de sa valeur ajoutée organisationnelle. Le géomaticien passe du statut d'opérateur d'exécution (saisie cartographique, numérisation, lancement manuel de boîtes d'outils) à celui d' **architecte de politiques géospatiales, de concepteur de garde-fous d'IA et de superviseur de systèmes autonomes** . L'humain conserve la responsabilité stratégique de la gouvernance, valide les arbitrages territoriaux complexes et supervise les exceptions remontées par les agents.

ÉVOLUTION DU RÔLE DU GÉOMATICIEN

RÔLE TRADITIONNEL	RÔLE AGENTIQUE MODERNE
- Saisie de données & vectoriel	- Architecte de politiques SIG
- Lancement manuel d'outils SIG	- Concepteur de garde-fous IA
- Traitement répétitif	- Superviseur d'agents autonomes

Le déploiement d'une équipe de géomaticiens autonomes au sein d'une organisation s'articule autour d'une feuille de route opérationnelle en 5 étapes clés :

Provisionnement de l'infrastructure VPS/Docker sécurisée : Instancier un serveur dédié (ex. VPS Linux sous Docker), configurer le proxy inverse avec chiffrement TLS, activer le serveur API (API_SERVER_ENABLED=true), définir une clé d'authentification robuste (API_SERVER_KEY) et fixer le port d'écoute sur 8642.

Configuration du moteur d'agent, du LLM et des garde-fous de stockage : Raccorder la clé d'API (OpenRouter ou Nous Portal), sélectionner un modèle optimisé pour l'usage d'outils et le cache (DeepSeek-V4-Flash), configurer la persistance contextuelle via X-Hermes-Session-Key, et activer le plafonnement des sorties d'outils en historique (gateway.api_server.history_tool_output_max_chars: 2000).

Connexion des identités machines, initialisation du réservoir de compétences et branchement a l'assistant : Attribuer des identités machines à moindre privilège pour chaque agent. **Avant d'autoriser la création autonome de compétences** , pré-charger l'environnement avec un socle de compétences de base certifiées (validation topologique, conversion de projections, contrôle de qualité attributaire) conformes à la norme agentskills.io au sein du *Skills Hub* . Interconnecter ensuite les connecteurs Python/PyGIS et les boîtes d'outils.

Implémentation des garde-fous de gouvernance et des files d'approbation : Mettre en place la validation déterministe des commandes au niveau de la passerelle d'orchestration (externe au LLM), paramétrer les seuils de risque et router toute action modifiant des tables en production vers la file d'approbation humaine (/v1/runs/{run_id}/approval).

Mise en place des canaux de communication inter-plateformes et des tâches Cron : Interconnecter l'agent aux outils de communication de l'équipe (Telegram, Teams, Slack), désigner le canal principal de notification (/set home) et programmer les automatisations d'arrière-plan récurrentes via l'API /api/jobs.

L'impact organisationnel de cette transformation est immédiat : l'automatisation des tâches répétitives de géotraitement permet une accélération massive des études territoriales tout en maintenant une qualité de donnée optimale grâce à des audits continus.

En conclusion, la transition vers une géomatique autonome propulsée par l'IA agentique repose sur l'équilibre parfait entre trois piliers indissociables : un moteur d'agent persistant et conteneurisé (*Runtime*), un outillage spatial de pointe, et une gouvernance déterministe native (*Gouvernance d'Exécution*). En maîtrisant cette chaîne d'ingénierie, les services géospatiaux se dotent d'une force de travail hybride, hautement évolutive, sécurisée et à forte valeur ajoutée au service de la décision territoriale.

Tableau 14.1 Référentiels normatifs et leur apport à l'architecture.

Référentiel	Objet	Apport à l'architecture
ISO 19157	Qualité de l'information géographique (précision positionnelle, intégrité topologique)	Garantir la qualité des jeux de données utilisés par l'IA
OGC (WMS, WFS, WCS, GeoJSON)	Interopérabilité des services géospatiaux	Structuration uniforme de l'acquisition de données
Charte Locus	Minimisation et limitation des usages secondaires	Préservation de la géo-confidentialité

Conclusion

Redéfinition du rôle du géomaticien

L'avènement de l'IA agentique ne marque pas la disparition du géomaticien humain, mais opère une élévation majeure de sa valeur ajoutée organisationnelle. Le géomaticien passe du statut d'opérateur d'exécution (saisie cartographique, numérisation, lancement manuel de boîtes d'outils) à celui d'architecte de politiques géospatiales, de concepteur de garde-fous d'IA et de superviseur d'agents autonomes. L'humain conserve la responsabilité stratégique de la gouvernance, valide les arbitrages territoriaux complexes et supervise les exceptions remontées par les agents.

Tableau C.1 Évolution du rôle du géomaticien.

Rôle traditionnel	Rôle agentique moderne
Saisie de données & vectoriel	Architecte de politiques SIG
Lancement manuel d'outils SIG	Concepteur de garde-fous IA
Traitement répétitif	Superviseur d'agents autonomes

Perspectives futures et recommandations operationnelles

L'introduction des technologies agentiques et des environnements de développement assistés par l'IA redefinit la pratique de la géomatique. L'automatisation integrale n’est plus une perspective lointaine, ces technologies permettent d'optimiser l'exécution des flux de travail quotidiens et de democratiser l'exploitation des donnees spatiales.

perspectives d'ingénierie : Quatre axes de developpement operationnels

- ➔ Architectures hybrides : articuler l'assistant de bureau (A) et le service agentique (B) via une API, avec communication contrôlée.
- ➔ Ingénierie logicielle rigoureuse : structures de répertoires standardisées, spécifications en amont, règles de style persistantes.
- ➔ Filtres d'intégrité géométrique : renforcer les contrôles topologiques et CRS en amont du calcul géospatial.
- ➔ Maintien de l'expertise humaine : conserver le regard critique du géomaticien comme pivot des processus décisionnels spatiaux.

Pour les organisations engagées dans la transformation de leurs processus géomatiques, plusieurs axes de développement opérationnels se dégagent de l'analyse des technologies actuelles :

La sécurisation des analyses spatiales repose sur le déploiement d'architectures hybrides associant la flexibilité du langage naturel à la rigueur d'outils géomatiques déterministes. L'exposition de workflows de traitement validés sous la forme de services MCP normalisés offre aux agents IA un cadre d'exécution sécurisé et performant. Cette approche évite de confier la rédaction de code spatial complexe à des modèles sujets aux hallucinations logiques, en favorisant l'appel de procédures de calcul standardisées au sein d'entrepôts de données ou de bases de données relationnelles structurées.

La fiabilisation des développements assistés par l'IA exige l'adoption de pratiques d'ingénierie logicielle rigoureuses au sein des équipes de développement géospatial. La mise en place systématique de structures de répertoires standardisées, la rédaction rigoureuse de spécifications techniques en amont du codage, et la configuration de règles de style persistantes (CLAUDE.md, .cursor/rules/) s'avèrent indispensables pour optimiser l'utilisation de la mémoire de contexte des agents conversationnels. Ces pratiques limitent la dérive du code source et garantissent la maintenabilité des applications d'entreprise.

L'implémentation de filtres d'intégrité géométrique constitue une protection essentielle pour sécuriser l'exécution des modèles générés de manière autonome. L'intégration de couches d'analyse et de validation statique en amont du calcul géospatial permet de détecter précocement les anomalies de planification logique, telles que les inversions d'opérations géométriques ou l'absence de transformation de projection (CRS), évitant ainsi la consommation de ressources de calcul cloud coûteuses pour des livrables erronés.

Enfin, l'expertise humaine doit être maintenue comme pivot des processus décisionnels spatiaux. Si l'IA s'établit comme un assistant efficace pour automatiser l'acquisition de données, nettoyer les tables d'attributs ou rédiger des scripts de traitement complexes, le regard critique du géomaticien demeure incontournable pour valider la pertinence méthodologique des modèles, contrôler l'intégrité scientifique des analyses géostatistiques et garantir l'éthique de la représentation cartographique de l'espace terrestre.

Vers l'architecture B service géomatique agentique

L'évolution vers un service d'orchestration (Architecture B) destiné à exécuter des tâches longues, planifiées ou concurrentes, avec isolation, files d'approbation, journaux et supervision constitue une perspective majeure. Elle exige un socle infrastructurel conteneurisé, une gouvernance déterministe native et une traçabilité immuable. Elle ouvre sur l'orchestration multi-agents, la création autonome de compétences et l'automatisation planifiée, mais introduit des enjeux de sécurité qui nécessitent une attention particulière.

Perspective finale

La transition vers une géomatique autonome propulsée par l'IA agentique repose sur l'équilibre entre trois piliers indissociables : un moteur d'agent fiable, un outillage spatial rigoureux, et une gouvernance déterministe native. La maîtrise de cette chaîne d'ingénierie permet de doter les services géospatiaux d'une force de travail hybride, évolutive, sécurisée et à forte valeur ajoutée au service de la décision territoriale.

Bibliographie et référentiels

B.1 Publications et benchmarks académiques

- **GISAgentBench : A Practitioner-Sourced Benchmark for Evaluating LLM Agents on GIS Tasks.** arXiv:2608.01645. Benchmark de 349 tâches SIG multi-étapes, 128 API, six zones géographiques. .
- **Chen, M. et al. (2021).** Evaluating Large Language Models Trained on Code (pass@k). OpenAI. Métrique pass@k..
- **Benchmark SWE-bench.** Strict Task Success Rate (TSR) pour agents multi-étapes. [À SOURCER].
- **Agence internationale de l'énergie (IEA).** Electricity 2024. https://www.iea.org/reports/electricity-2024 Consommation énergétique du numérique.
- **Feng, S., McDonald, K. A., & Zhang, L. GIScience** in the era of Artificial Intelligence: A Research Agenda towards Autonomous GIS. [arXiv / USGS — à compléter par revue, volume, DOI]. Cadre « 5 objectifs / 5 niveaux / 5 fonctions » de l'Autonomous GIS.

B.2 Standards et protocoles

- **Model Context Protocol (MCP).** Anthropic, fin 2024. Standard ouvert d'exposition d'outils aux LLM.].
- **Open Geospatial Consortium (OGC).** Standards WMS, WFS, WCS, GeoJSON. Interopérabilité des services géospatiaux.
- **LLM-Find.** Framework d'approvisionnement autonome en données géospatiales basé sur des handbooks. [À SOURCER publication/documentation].
- **GIS Stack Exchange.** Plateforme de questions-réponses source des tâches GISAgentBench.

B.3 Référentiels normatifs et réglementaires

- **Règlement européen sur l'intelligence artificielle (EU AI Act).** Approche par les risques. [numérotation des articles 5, 14, 50 et champ d'application].
- **Règlement Général sur la Protection des Données (RGPD).** UE, 2018.
- **ISO 19157.** Information géographique Qualité des données géographiques.
- **NIST AI RMF.** Artificial Intelligence Risk Management Framework (profil IA générative).
- **OWASP.** Top 10 pour l'IA agentique (sur autorisation, détournement d'outils, injections indirectes).
- **Charte Locus (Locus Charter).** Minimisation de la collecte de données de localisation..
- **UN-GGIM.** Comité d'experts de l'ONU sur la gestion de l'information géospatiale.
- [1] AIE, Energy and AI (avril 2025), résumé : https://www.iea.org/reports/energy-and-ai/executive-summary · communiqué : https://www.iea.org/news/ai-is-set-to-drive-surging-electricity-demand-from-data-centres-while-offering-the-potential-to-transform-how-the-energy-sector-works
- [2] Luccioni, Jernite, Strubell, « Power Hungry Processing: Watts Driving the Cost of AI Deployment? », ACM FAccT 2024, doi 10.1145/3630106.3658542 : https://arxiv.org/abs/2311.16863
- [2bis] Échange public avec l'équipe AI Energy Score (Hugging Face) sur la variabilité des mesures : https://huggingface.co/spaces/AIEnergyScore/Leaderboard/discussions/3 (source communautaire, poids limité)
- [3] Mistral AI, « Notre contribution à un standard environnemental mondial pour l'IA » (22 juillet 2025) : https://mistral.ai/fr/news/our-contribution-to-a-global-environmental-standard-for-ai/

- [3bis] Solutions Numériques, reprise des chiffres de l'étude : https://www.solutions-numeriques.com/?p=240044
- [3ter] Next.ink, analyse de l'étude (part du matériel, limites) : https://next.ink/193277/ia-mistral-apporte-de-la-transparence-sur-limpact-environnemental-dun-modele/
- [5] UNESCO, Recommandation sur l'éthique de l'IA (2021) : https://www.unesco.org/en/artificial-intelligence/recommendation-ethics · AlgorithmWatch (caractère non contraignant) : https://algorithmwatch.org/en/unesco-adopts-recommendation-on-the-ethics-of-ai/
- [6] Conseil de l'Europe, ouverture à la signature de la Convention-cadre : https://www.coe.int/en/web/portal/-/council-of-europe-opens-first-ever-global-treaty-on-ai-for-signature
- [6bis] Conseil de l'Europe, signatures du Canada et du Japon : https://www.coe.int/web/portal/-/canada-and-japan-sign-council-of-europe-s-first-ever-global-treaty-on-ai
- [7] Commission européenne, AI Act Service Desk, calendrier de mise en œuvre : https://ai-act-service-desk.ec.europa.eu/en/ai-act/timeline/timeline-implementation-eu-ai-act
- [7bis] Texte du règlement (UE) 2024/1689 : https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng
- [8] Analyses du Digital Omnibus (secondaires, à recouper avec EUR-Lex) : Atias Avocats, https://atiasavocats.com/calendrier-ai-act-echeances-application/ · iones.io, https://iones.io/insights/ai-act-obligations-aout-2026-omnibus/ · Adevweb (sanctions), https://www.adevweb.com/ressources/ai-act-entreprise-2026
- [9] Bundesnetzagentur, « Energy and resource efficiency » (articles 40, 53, 95 de l'AI Act) : https://www.bundesnetzagentur.de/EN/Areas/Digitalisation/AI/12_EnergyEfficiency/start.html
- [9bis] Arcep, contribution à la consultation de la Commission européenne (nov. 2025) : https://www.arcep.fr/uploads/tx_gspublication/Arcep-contribution-EC-public-consultation_Strategic-Roadmap-digitalisation-AI-energy-sector_nov2025.pdf
- [9ter] Analyse académique des limites environnementales de l'AI Act : https://arxiv.org/html/2409.07471v1 · Fondation Heinrich Böll : https://pl.boell.org/en/2024/04/08/eu-ai-act-and-environmental-protection-case-missed-opportunity
- [10] Suivi des lois d'État américaines (secondaire) : https://www.glacis.io/guide-state-ai-laws · https://superintelligencecompliance.ai/guides/us-state-ai-laws

B.4 Concepts scientifiques

- **MAUP (Modifiable Areal Unit Problem).** Problème de l'unité spatiale modifiable biais d'agrégation spatiale. [À SOURCER référence fondatrice,Openshaw].
- **Géo-confidentialité (Geo-privacy).** Protection des données de localisation.
- **GeoAI Ethics.** Éthique de l'intelligence artificielle géographique.

B.5 Outils et écosystèmes logiciels

- **QGIS / PyQGIS / GDAL / PostGIS.** Logiciels et bibliothèques de géotraitement.
- **Ollama, LM Studio.** Écosystèmes d'inférence locale. [VÉRIFIER versions observées et dates].
- **llama.cpp, format GGUF.** Moteur et format de quantification (Q4_K_M, Q8_0).

- **Hermes Agent (Nous Research).** Runtime d'agent open-source. [À SOURCER documentation technique et version].
- **Shapely, PyProj, Rasterio, PySAL.** Bibliothèques Python de calcul géospatial rigoureux.
- **MapSmith.** Framework de validation statique de graphes d'exécution. [À SOURCER].

B.6 Note sur la vérifiabilité

Engagement de traçabilité

Chaque affirmation chiffrée du manuscrit est accompagnée d'un marqueur [À SOURCER] ou [VÉRIFIER] lorsqu'elle ne peut être rattachée à une source confirmée. Avant soutenance, le candidat complétera chaque entrée marquée par sa référence bibliographique complète (auteur, titre, année, éditeur/revue, DOI ou URL) et vérifiera les chiffres et versions cités à la date d'observation.

ANNEXES

Annexe 1: Architecture Multi IA

Architecture de l'assistant géomatique local dans QGIS : modèles, RAG et outils

Le projet consiste à créer un assistant IA intégré à QGIS, capable de comprendre une demande en langage naturel, de retrouver les informations utiles, puis de préparer ou d'exécuter des traitements SIG. L'idée centrale est de ne pas demander à un seul modèle de tout faire : chaque composant a un rôle précis un modèle rapide pour comprendre, un système RAG pour rechercher dans la documentation, un modèle spécialisé pour les demandes complexes, et des outils Python pour agir réellement dans QGIS.

A.1 Vue d'ensemble

1. Utilisateur dans QGIS

    « Fais un buffer de 100 m autour de l'assainissemet »

2. `geoai_qgis.py`

    Orchestre les étapes, récupère les couches et choisit le traitement adapté.

3A. Gemma 4B

    Analyse rapide de la demande, extraction de l'opération et des paramètres.

    Traitements simples

3B. RAG + Gemma-Geo

    Recherche documentaire et raisonnement pour les demandes complexes.

    Traitements complexes

4. Outils Python et API QGIS

    Recherche de couches, algorithmes Processing, lecture des attributs, contrôles et création de résultats.

5. Validation et résultat dans QGIS

    Présentation de l'action, confirmation si nécessaire, exécution et ajout de la couche au projet.

Les modèles sont exécutés localement avec Ollama, accessible depuis QGIS à l'adresse `http://127.0.0.1:11434`. Le script `geoai_qgis.py` pilote le processus. Le serveur `geoai_server.py` n'est pas nécessaire dans l'architecture actuelle.

A2. Le rôle de chaque modèle

Rapide

Gemma 4B `gemma4:latest`

C'est le premier niveau d'analyse. Il transforme la phrase de l'utilisateur en informations exploitables par Python.

Exemple de résultat attendu :

```
{
  "operation": "buffer",
  "parametres": {
    "couche": "assainissemet",
    "distance": 100
  }
}
```

Il n'a pas besoin de produire une longue explication. Il doit surtout identifier l'opération, les paramètres et les éléments qui manquent.

Recherche

EmbeddingGemma `embeddinggemma:latest`

Ce modèle sert à transformer les textes en vecteurs numériques, appelés embeddings. Ces vecteurs permettent de retrouver les documents dont le sens est proche d'une question.

Il ne répond pas lui-même à la question et n'exécute pas de traitement QGIS. Son rôle est d'aider le RAG à retrouver les passages pertinents.

Spécialisé

Gemma-Geo `gemma-geo:latest`

Il intervient lorsque la demande nécessite davantage de raisonnement : plusieurs opérations successives, interprétation de règles métier, analyse d'un problème SIG ou élaboration d'un plan de traitement.

Il reçoit la question, l'analyse initiale, le contexte du projet QGIS et les documents récupérés par le RAG. Il peut alors produire une réponse ou une proposition d'action structurée.

Pourquoi cette séparation ? Tu as constaté que Gemma 4B répond plus vite, tandis que Gemma-Geo peut être lent, même pour un simple buffer. Le routage évite donc de solliciter le modèle spécialisé lorsque Python et QGIS savent déjà effectuer le travail.

A.3 Le module RAG

Le RAG (Retrieval-Augmented Generation) permet à l'IA de s'appuyer sur les documents de l'utilisateur plutôt que de répondre uniquement à partir de son entraînement :

- **Préparation :** PDF, DOCX, guides QGIS, procédures métier, documentation SIG.
- **Découpage :** segmentation en passages assez courts pour être recherchés efficacement.
- **Vectorisation :** conversion en embeddings via EmbeddingGemma, puis indexation.
- **Recherche :** la question est vectorisée et comparée à l'index ; passages pertinents récupérés.

- **Enrichissement :** passages retenus transmis au modèle de raisonnement avec la question et le contexte QGIS.

> **Limite du RAG**
> Le RAG n'est pas une base de données qui connaîtrait automatiquement tous les fichiers. Les documents doivent être ingérés et indexés, et la recherche doit récupérer les bons passages. Le RAG ne garantit pas à lui seul l'exactitude de la réponse. Dans le projet, rag.py gère cette partie (création des embeddings et recherche documentaire).

Exemple concret

« Quelles vérifications faut-il faire avant d'importer un réseau d'eau dans EPANET ? »

1. Gemma 4B identifie qu'il s'agit d'une question technique.
2. Le RAG recherche les passages pertinents dans tes procédures et documents.
3. Gemma-Geo reçoit ces passages et rédige une réponse fondée sur les informations récupérées.
4. L'IA peut expliquer les contrôles à réaliser : géométrie, connectivité, attributs, unités et cohérence des données, si ces points sont étayés par les sources disponibles.

Point important : le RAG n'est pas une base de données qui connaît automatiquement tous tes fichiers. Il faut que les documents soient ingérés et indexés, et que la recherche récupère les bons passages. Il ne garantit pas à lui seul l'exactitude de la réponse.

Dans ton projet, `rag.py` gère cette partie, notamment la création des embeddings et la recherche documentaire.

A.4 La couche d'outils

C'est une distinction fondamentale : un modèle de langage ne modifie pas directement le projet QGIS. Il interprète une demande et peut proposer une action ; le code Python appelle ensuite les fonctions réelles.

Dans ton architecture, les outils sont principalement des fonctions Python qui utilisent l'API PyQGIS et les algorithmes Processing.

Outil ou fonction	Rôle
`get_qgis_context()`	Récupère des informations sur le projet et ses couches.
`filtrer_contexte_qgis()`	Réduit le contexte transmis au modèle pour éviter un prompt trop volumineux.

`trouver_couche_qgis()`	Recherche une couche, y compris lorsque son nom est mal orthographié.
`rechercher_rag()`	Récupère les passages documentaires pertinents.
`executer_buffer()`	Exécute un tampon avec `native:buffer`.
`executer_clip()`	Exécute une découpe avec `native:clip`.
`executer_resultat()`	Interprète le résultat structuré et oriente vers la fonction d'exécution appropriée.

Les outils peuvent ensuite être étendus à d'autres opérations : reprojection, intersection, sélection spatiale, jointure, contrôle topologique, export ou calcul de champs.

Exemple : retrouver une couche malgré une faute

L'utilisateur écrit :

> Fais un buffer de 100 m autour de l'assainissemet.

Gemma 4B extrait le nom fourni par l'utilisateur : `assainissemet`.

Ensuite, `trouver_couche_qgis()` recherche ce nom parmi les couches du projet. Dans ton code, cette recherche utilise notamment `difflib.get_close_matches()` pour comparer les noms proches.

Le script récupère alors le nom exact de la couche QGIS et prépare une action structurée :

```
{    "type": "action",    "action": "buffer",    "parameters": {        "layer": "nom_exact_de_la_couche",
"distance": 100,        "segments": 5,        "dissolve": False,        "output":
"buffer_nom_exact_de_la_couche"    }}
```

Ce dictionnaire est un contrat entre l'analyse de l'IA et le code Python. Il faut vérifier ses paramètres avant l'exécution : l'IA peut extraire une mauvaise distance ou se tromper sur la couche.

Le bouton d'exécution de ton interface peut ensuite demander confirmation et lancer le traitement Processing. QGIS crée le résultat et l'ajoute au projet.

Attention : ce mécanisme est un système d'outils Python piloté par le modèle. Ce n'est pas nécessairement un système de tool calling natif où le modèle choisit et appelle lui-même des fonctions via un protocole dédié.

A.5 Parcours d'une demande

Voici les deux parcours principaux de ton architecture.

Parcours A Opération simple

1. L'utilisateur demande un buffer de 100 m.
2. Gemma 4B extrait `buffer`, la couche recherchée et la distance.
3. Python retrouve la couche exacte dans QGIS.
4. Python prépare l'action sans appeler Gemma-Geo.
5. L'utilisateur vérifie la proposition et confirme.
6. PyQGIS exécute `native:buffer`, puis ajoute la sortie au projet.

Parcours B Question ou opération complexe

1. Gemma 4B analyse rapidement la demande.
2. Python récupère le contexte utile du projet QGIS.
3. Le RAG recherche les passages pertinents dans la documentation.
4. Gemma-Geo reçoit la question, l'analyse, le contexte et les documents.
5. Gemma-Geo formule une réponse ou propose un plan d'action.
6. Python valide la structure de l'action, puis demande confirmation avant l'exécution si nécessaire.

Si une information manque par exemple la distance du buffer Python peut poser une question à l'utilisateur au lieu de laisser le modèle inventer une valeur. Pour gérer les réponses comme « 100 m » ou « prends la deuxième couche », il faudra conserver l'historique des échanges dans l'interface QGIS et réinjecter ce contexte dans l'analyse suivante.

A.6 Localisation des données et des traitements

Il faut distinguer trois sources de connaissances :

- Les documents du RAG : procédures, PDF, DOCX et documentation que tu as indexés.
- Le contexte QGIS : couches présentes dans le projet, noms, types de géométrie, CRS et informations que ton script choisit de transmettre.
- Les données SIG elles-mêmes : géométries, attributs, tables et rasters. Elles sont lues et modifiées par QGIS ou ses fournisseurs de données ; elles ne sont pas automatiquement entièrement envoyées au modèle.

Cette séparation est utile pour éviter d'envoyer des milliers de couches ou des données inutiles à l'IA. Le contexte doit être filtré selon la demande.

Risque	Contrôle à prévoir
Mauvaise couche identifiée	Recherche approximative, candidats en cas d'ambiguïté et confirmation.
Distance ou paramètre incorrect	Vérification des valeurs extraites et affichage avant exécution.
Réponse documentaire non fondée	Fournir les passages récupérés et distinguer les faits documentés des déductions.
Action mal formée	Validation stricte du JSON et liste blanche d'opérations autorisées.
Traitement incompatible	Vérification du type de géométrie, du CRS, des champs et des paramètres.
Modification involontaire du projet	Prévisualisation, confirmation et gestion des erreurs.
Conversation interrompue	Historique de session pour les demandes de clarification.

A.7 Synthèse de l'architecture actuelle

Tableau A.1 Synthèse des composants de l'architecture actuelle.

Composant	Rôle
Ollama	Exécute les modèles locaux et expose l'API utilisée par QGIS.
Modèle d'analyse rapide (Gemma 4B)	Extraction des paramètres pour les opérations simples.
EmbeddingGemma + RAG	Recherche sémantique dans les documents indexés.
Modèle de raisonnement (Gemma-Geo)	Raisonnement et demandes SIG complexes, lorsque le routage le sollicite.
Python / PyQGIS	Recherche des couches, contrôles, appels Processing et exécution effective.

L'architecture cible est donc une IA locale hybride avec RAG et outils : les modèles comprennent et raisonnent, le RAG apporte des connaissances documentaires, et Python garde la maîtrise des interactions avec QGIS.

Annexe 2 scénario d'exécution et pseudo-flux d'orchestration

Scénario illustratif

L'utilisateur demande : « Examiner les croisements entre les routes et les cours d'eau dans l'emprise actuellement visible, puis préparer un résultat consultable dans QGIS. » Cet exemple ne suppose ni données particulières, ni nombre d'entités, ni distance, ni résultat géographique réel.

1. QGIS transmet la demande et le contexte autorisé : couches Routes et Cours d'eau disponibles, emprise courante, systèmes de coordonnées et droits applicables.
2. L'orchestrateur vérifie que l'emprise n'est pas vide et que les deux couches sont accessibles. Il signale toute incompatibilité de CRS ou de géométrie avant de poursuivre.
3. Le RAG récupère la méthode approuvée décrivant le contrôle d'intersection entre les deux types de couches, ainsi que les règles de qualité applicables.
4. Modèle (routage Gemma) propose un plan structuré : harmoniser le contexte spatial si nécessaire, contrôler les géométries, calculer les intersections avec l'outil autorisé et produire une couche ou un rapport.
5. L'orchestrateur vérifie que l'outil d'intersection existe dans le catalogue, que ses paramètres sont complets et que la sortie ne remplacera pas une donnée existante sans autorisation.
6. Si la politique l'exige, QGIS demande une confirmation avant l'exécution. La confirmation est liée à l'emprise, aux couches, à l'outil et aux paramètres présentés.
7. La couche d'outils exécute l'opération autorisée via l'interface prévue, journalise les validations et contrôle l'intégrité de la sortie.
8. Le résultat est renvoyé vers QGIS avec son statut, ses avertissements, les couches utilisées, la méthode citée et les contrôles effectués.

Pseudo-flux d'orchestration lisible

Début

Recevoir demande + contexte QGIS

→ Normaliser et vérifier les entrées

→ Si contexte insuffisant : demander une précision et arrêter l'exécution

→ Rechercher documentation, méthode et script autorisés

→ Générer interprétation + plan structuré avec Modèle (routage Gemma)

→ Vérifier outil, paramètres, permissions et limites

→ Si risque élevé : demander confirmation humaine

→ Exécuter uniquement l'outil enregistré

→ Contrôler sortie, qualité et statut

→ Journaliser action et références

→ Restituer résultat, avertissements et citations dans QGIS

Fin

Résultats possibles du scénario

- **Succès** une sortie conforme est disponible dans QGIS et les sources utilisées sont citées.
- **Succès avec avertissement** la sortie est disponible mais certaines géométries ou validations nécessitent une revue.
- **Demande de précision: l'emprise**, les couches ou la règle de traitement ne sont pas suffisamment déterminées.
- **Refus contrôlé** l'outil, l'accès, le paramètre ou l'opération demandée n'est pas autorisé.
- **Échec** l'exécution n'a pas produit de résultat exploitable ; le message d'erreur et l'identifiant de traçabilité sont retournés sans présenter de sortie invalide comme un résultat.